

DOCUMENTACIÓN TÉCNICA COMPLETA

Sistema Web Interno de Punto de Venta y Gestión Comercial

MVP — Ventas, Inventario, Cotizaciones, Pagos, Reportes y Contabilidad Básica

Dato	Valor
Versión	1.0
Fecha	Julio 2026
Documento fuente	Documento Maestro POS / Gestión Comercial MVP v1.0
Tipo de sistema	Plataforma web interna de gestión comercial / POS
Rubro objetivo	Productos técnicos, equipos, repuestos y maquinaria
Stack	Angular · NestJS · PostgreSQL · Prisma · Docker
Arquitectura	Monolito modular con Clean Architecture ligera
Estado	Base técnica aprobada para iniciar desarrollo

Documento interno de trabajo — demo sin datos reales ni branding de una empresa específica

1. Índice de contenido

El índice se actualiza en Word con: Referencias → Actualizar tabla (o clic derecho → Actualizar campos).

2. Control de versión del documento	4
3. Resumen ejecutivo técnico	4
4. Objetivo técnico del sistema	4
5. Alcance técnico del MVP	5
6. Fuera de alcance del MVP	5
7. Arquitectura general recomendada	6
8. Justificación de la arquitectura monolítica modular con Clean Architecture ligera	6
9. Diagrama de arquitectura en texto	6
10. Diagrama de arquitectura en Mermaid	7
11. Stack tecnológico recomendado	7
12. Estructura recomendada del backend	8
13. Estructura recomendada del frontend	8
14. Estructura recomendada de base de datos	9
15. Modelo entidad-relación inicial	9
16. Diagrama ERD en Mermaid	10
17. Descripción de entidades principales	10
18. Tablas sugeridas: campos, tipos, claves y restricciones	11
19. Relaciones entre tablas	21
20. Flujos técnicos por módulo	21
21. Diagramas de flujo Mermaid para procesos principales	23
22. Reglas de negocio por módulo	24
23. Validaciones funcionales y técnicas	26
24. API endpoints sugeridos por módulo	26
25. Contratos request/response JSON para endpoints principales	29
26. Manejo de autenticación JWT	31
27. Manejo de roles y permisos	31
28. Estrategia de seguridad	32
29. Estrategia de auditoría	32
30. Estrategia de manejo de errores	33
31. Estrategia de logging	33
32. Estrategia de transacciones de base de datos	33
33. Estrategia para ventas y reversión de stock	34
34. Estrategia para pagos completos y parciales	34
35. Estrategia para contabilidad básica / precontabilidad	34
36. Estrategia de correlativos	35

37. Estrategia de reportes y dashboard	35
38. Variables de entorno necesarias	36
39. Plan de migraciones inicial	37
40. Datos semilla para demo	37
41. Plan de desarrollo por fases	38
41.1 Tabla resumen del plan (Fase · Módulo · Actividad · Prioridad · Dependencia · Resultado · Estado)	39
42. Backlog técnico priorizado	40
43. Historias técnicas por sprint	41
44. Criterios de aceptación técnicos	42
45. Checklist para iniciar desarrollo	43
46. Checklist para probar la demo.....	43
47. Riesgos técnicos y funcionales	44
48. Recomendaciones para mockups y pantallas.....	45
49. Recomendación final de cómo empezar a programar	45
50. Anexos técnicos	46

2. Control de versión del documento

Versión	Fecha	Autor	Descripción del cambio	Estado
1.0	Julio 2026	Equipo técnico	Emisión inicial de la documentación técnica completa, derivada del Documento Maestro v1.0.	Vigente

Dato	Valor
Documento fuente	Documento Maestro POS / Gestión Comercial MVP v1.0
Tipo de documento	Documentación técnica de desarrollo
Audiencia	Equipo de desarrollo, líder técnico y gerencia del proyecto
Alcance temporal	MVP demo (sin datos reales ni branding de empresa específica)
Regla de precedencia	Ante cualquier duda, prevalecen las decisiones del Documento Maestro

3. Resumen ejecutivo técnico

Este documento define la base técnica completa para construir el MVP de un sistema web interno de punto de venta y gestión comercial, orientado a una empresa del rubro de productos técnicos, equipos, repuestos y maquinaria. El sistema no es un POS genérico de minimarket ni un ERP completo: es una solución interna de gestión comercial con catálogo técnico, cotizaciones, ventas, pagos completos y parciales, inventario por movimientos, reportes gerenciales, precontabilidad automática y auditoría de acciones críticas.

La solución se implementará como un monolito modular con Clean Architecture ligera, usando Angular en frontend, NestJS en backend, PostgreSQL como base de datos, Prisma como ORM y Docker para despliegue reproducible. La autenticación se realizará con JWT en cookie HttpOnly y la autorización por roles se validará siempre en backend.

Quedan explícitamente fuera del MVP: alquiler de maquinaria, facturación electrónica SUNAT, compras y proveedores, kardex valorizado, multi-almacén, multi-moneda y contabilidad tributaria completa. La contabilidad incluida es precontabilidad básica automática que no reemplaza un software contable formal.

4. Objetivo técnico del sistema

Construir una plataforma web interna, segura y transaccional que permita:

- Administrar un catálogo técnico de productos, repuestos, equipos y servicios no inventariables, con especificaciones flexibles e imágenes.
- Registrar clientes y prospectos (personas y empresas) con control de duplicidad por documento.
- Generar cotizaciones comerciales con estados, vigencia, PDF simple y conversión a venta.
- Registrar ventas directas desde POS o desde cotización, con descuento de stock transaccional y nota de venta interna no fiscal.
- Registrar pagos completos y parciales con control de saldo y cuentas por cobrar.
- Controlar inventario exclusivamente por movimientos trazables, sin stock negativo.
- Generar asientos precontables automáticos cuadrados (debe = haber) por ventas, pagos y anulaciones.
- Entregar dashboard por rol y diez reportes básicos para decisión gerencial.
- Registrar auditoría inmutable de acciones críticas.

Todo lo anterior con calidad de MVP serio: código mantenible, transacciones correctas, validación en backend y una demo presentable, evitando complejidad prematura.

5. Alcance técnico del MVP

El MVP incluye los siguientes módulos, todos obligatorios según el Documento Maestro:

Módulo	Descripción técnica	Prioridad
Autenticación	Login/logout con JWT en cookie HttpOnly, validación de estado ACTIVE, bloqueo por intentos fallidos y must_change_password.	Alta
Usuarios y roles	CRUD de usuarios, 4 roles precargados por seed (ADMIN, SELLER, WAREHOUSE, MANAGEMENT), reset de contraseña por administrador.	Alta
Dashboard	Indicadores por rol tras el login: ventas, cobros, stock bajo, cotizaciones y cuentas por cobrar.	Alta
Productos	Catálogo con SKU único, tipo PRODUCT/SERVICE, is_inventory_tracked, descripción comercial y observación interna separadas, especificaciones e imágenes.	Alta
Categorías	Jerarquía en una tabla con parent_id, sin ciclos, baja lógica.	Alta
Unidades	Unidades con allow_decimal para validar cantidades enteras o decimales.	Alta
Inventario	Un almacén lógico, movimientos trazables con previous_stock/new_stock, ajustes con motivo obligatorio, sin stock negativo.	Alta
Clientes	PERSON/COMPANY, prospecto y cliente por customer_stage, cliente genérico "Público general" solo para ventas al contado.	Alta
Cotizaciones	Estados PENDIENTE/ACEPTADA/RECHAZADA/VENCIDA/CONVERTIDA, snapshot de ítems, PDF simple y conversión a venta.	Alta
Ventas / POS	Venta directa o desde cotización, correlativo NV, descuento de stock transaccional, anulación con reversa.	Alta
Pagos	Pagos completos/parciales, métodos definidos, anulación con motivo, recalculo de saldo.	Alta
Reportes	Diez reportes básicos con filtros, excluyendo anulados de totales.	Alta
Configuración	Empresa, moneda única, IGV configurable, correlativos y parámetros comerciales.	Alta
Contabilidad básica	Asientos automáticos por venta, pago y anulación; plan de cuentas básico; sin asientos manuales.	Media/Alta
Auditoría básica	audit_logs con metadata JSONB, solo lectura, acciones críticas.	Media

6. Fuera de alcance del MVP

Los siguientes elementos quedan explícitamente fuera del MVP y no deben implementarse ni "prepararse" con complejidad adicional. Solo se permiten decisiones de diseño que no bloqueen su incorporación futura:

- Alquiler de maquinaria: contratos, tarifas, entrega/devolución y penalidades.
- Facturación electrónica SUNAT y libros electrónicos PLE. El sistema emite nota de venta interna no fiscal.
- Compras y proveedores completos. El ingreso de mercadería se cubre con movimientos manuales de inventario.
- Kardex valorizado y costeo de inventario.
- Multi-almacén (el MVP usa un único almacén lógico: Almacén General).
- Multi-moneda con tipo de cambio (una moneda principal configurable).
- Contabilidad tributaria completa, asientos manuales, conciliación bancaria e integración con software contable externo.
- Detracciones, percepciones y retenciones.
- CRM avanzado y notificaciones automáticas por correo/WhatsApp.

- Matriz compleja de permisos editable desde interfaz.

7. Arquitectura general recomendada

Se adopta una arquitectura monolítica modular: un solo backend NestJS organizado en módulos de negocio independientes, un solo frontend Angular organizado por features y una única base de datos PostgreSQL. No se usan microservicios.

- Frontend Angular consume una API REST del backend NestJS mediante HTTPS, enviando la cookie HttpOnly con el JWT.
- Cada módulo backend expone controller (HTTP), service/casos de uso (lógica de negocio), repository (acceso a datos vía Prisma) y DTOs con validación.
- Los módulos críticos (ventas, pagos, inventario, contabilidad) implementan casos de uso explícitos que orquestan transacciones.
- Servicios transversales: AuditService, SequenceService (correlativos), AccountingService (asientos automáticos) e InventoryService (movimientos de stock), consumidos por los casos de uso dentro de la misma transacción.
- PostgreSQL garantiza consistencia con transacciones, restricciones CHECK, claves foráneas e índices únicos.

8. Justificación de la arquitectura monolítica modular con Clean Architecture ligera

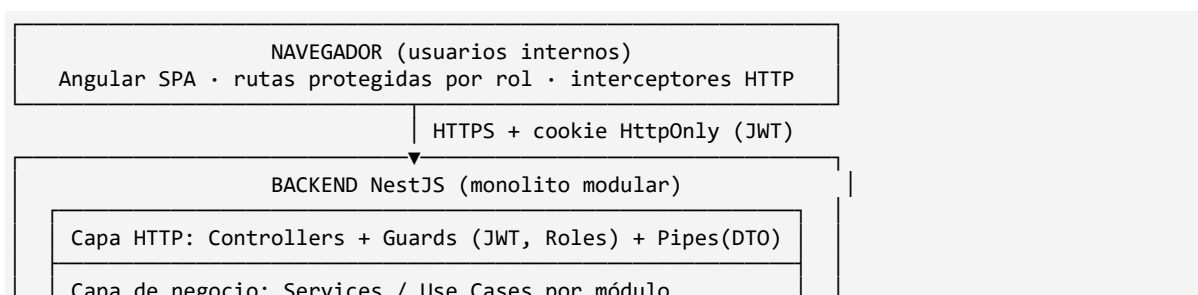
Por qué monolito modular y no microservicios

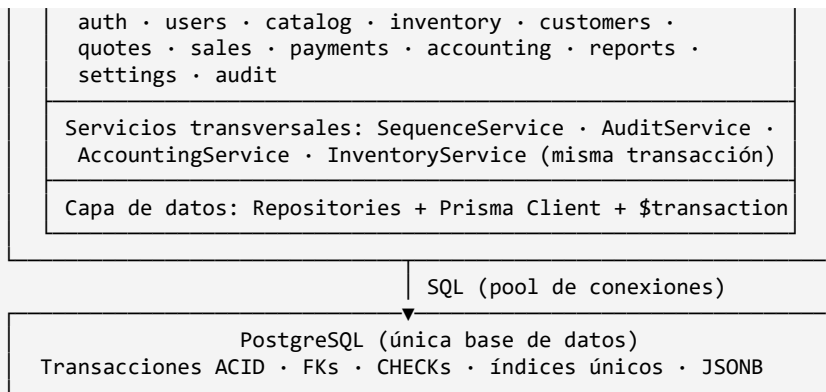
- El MVP tiene un solo equipo, un solo dominio de negocio y una sola base de datos: dividir en microservicios agregaría redes, colas, consistencia eventual y despliegues múltiples sin ningún beneficio inmediato.
- Las operaciones críticas (venta → stock → pago → asiento) requieren transacciones ACID en una sola base de datos; en microservicios eso exigiría sagas y compensaciones, complejidad injustificada para el alcance.
- Un monolito modular bien separado por módulos NestJS permite extraer servicios en el futuro si el negocio lo exige, sin pagar hoy el costo.

Por qué Clean Architecture "ligera"

- Se separa la lógica de negocio (use cases/services) del transporte HTTP (controllers) y del acceso a datos (repositories con Prisma), lo que facilita pruebas y mantenimiento.
- "Ligera" significa: sin capas ceremoniales innecesarias, sin interfaces duplicadas por cada clase ni mapeos redundantes. Solo los módulos críticos (ventas, pagos, inventario, contabilidad) usan casos de uso explícitos; los CRUD simples usan controller + service + repository.
- Las reglas de negocio viven en el backend; el frontend solo mejora la experiencia (ocultar botones no es seguridad).

9. Diagrama de arquitectura en texto





Archivos: almacenamiento local en demo → S3 compatible en producción.
Despliegue: Docker Compose (frontend, backend, base de datos).

10. Diagrama de arquitectura en Mermaid

El siguiente código Mermaid puede renderizarse en mermaid.live, en la wiki del repositorio o en la documentación del proyecto:

```

flowchart TB
    subgraph Client["Navegador - usuarios internos"]
        FE["Angular SPA<br/>features + guards por rol"]
    end
    subgraph API["Backend NestJS - monolito modular"]
        CTRL["Controllers + Guards JWT/Roles + DTO validation"]
        UC["Services / Use Cases por modulo"]
        CROSS["SequenceService · AuditService ·<br/>AccountingService · InventoryService"]
        REPO["Repositories + Prisma Client"]
        CTRL --> UC
        UC --> REPO
        UC --> CROSS
        CROSS --> REPO
    end
    DB["PostgreSQL<br/>transacciones ACID"]
    FILES["Archivos: local en demo,<br/>S3 compatible en produccion"]
    FE -- "HTTPS + cookie HttpOnly (JWT)" --> CTRL
    REPO --> DB
    UC --> FILES
  
```

11. Stack tecnológico recomendado

Capa	Tecnología	Versión sugerida	Motivo
Frontend	Angular + TypeScript	Angular 18+ LTS	Formularios reactivos, rutas protegidas, módulos por feature, tablas y dashboards para sistemas internos.
Backend	NestJS + TypeScript	NestJS 10+	Arquitectura modular con inyección de dependencias, guards, pipes e interceptores.
Base de datos	PostgreSQL	16	Consistencia transaccional, integridad referencial, JSONB para auditoría/especificaciones.
ORM	Prisma	5+	Migraciones versionadas, tipado end-to-end y \$transaction. Alternativa: TypeORM si el equipo lo domina.
Autenticación	JWT + cookie HttpOnly	passport-jwt	Sesión segura para web interna; token no accesible desde JavaScript.
Hashing	Argon2id (o bcrypt)	argon2	Contraseñas nunca en texto plano.
Validación	class-validator / class-transformer	—	Validación declarativa de DTOs en todos los endpoints.

Capa	Tecnología	Versión sugerida	Motivo
Documentación API	Swagger (@nestjs/swagger)	—	Contratos visibles y probables en desarrollo.
Archivos	Local (demo) / S3 compatible (producción)	—	Imágenes de productos, logo y PDFs de cotización.
PDF	pdfmake o Puppeteer (HTML→PDF)	—	Cotización y nota de venta interna imprimibles.
Despliegue	Docker + Docker Compose	—	Levantar frontend, backend y BD de forma reproducible.
Calidad	ESLint + Prettier + Jest	—	Estilo uniforme y pruebas de casos de uso críticos.

12. Estructura recomendada del backend

Backend NestJS organizado por módulos de negocio. Los módulos críticos incluyen carpeta use-cases:

```

backend/
├── prisma/
│   ├── schema.prisma
│   ├── migrations/
│   └── seed.ts
├── src/
│   ├── main.ts
│   ├── app.module.ts
│   ├── common/
│   │   ├── decorators/      (Roles, CurrentUser)
│   │   ├── guards/         (JwtAuthGuard, RolesGuard)
│   │   ├── filters/        (GlobalExceptionHandler)
│   │   ├── interceptors/   (LoggingInterceptor)
│   │   ├── dto/            (PaginationDto, IdParamDto)
│   │   ├── errors/         (AppException, códigos de error)
│   │   ├── config/         (env validation, configuration)
│   │   └── database/       (PrismaModule, PrismaService)
│   └── modules/
│       ├── auth/           (controller, service, strategies, dto)
│       ├── users/
│       ├── catalog/
│       │   ├── categories/
│       │   ├── units/
│       │   └── products/    (+ specifications, images)
│       ├── inventory/
│       │   └── use-cases/   (register-movement, revert-sale-stock)
│       ├── customers/
│       ├── quotes/
│       │   └── use-cases/   (create-quote, convert-quote-to-sale)
│       ├── sales/
│       │   └── use-cases/   (create-sale, cancel-sale)
│       ├── payments/
│       │   └── use-cases/   (register-payment, cancel-payment)
│       ├── accounting/
│       │   └── use-cases/   (create-entry-from-sale, -payment, -reversal)
│       ├── reports/
│       ├── dashboard/
│       ├── settings/       (company settings + sequences)
│       └── audit/          (AuditService + consulta)
├── test/
├── Dockerfile
└── .env.example

```

Convención por módulo: *.controller.ts (HTTP), *.service.ts o use-cases/*.use-case.ts (negocio), *.repository.ts (Prisma), dto/ (validación) y *.module.ts.

13. Estructura recomendada del frontend



Cada feature se carga con lazy loading y se protege con guards de autenticación y rol. Los componentes compartidos (buscador de productos, buscador de clientes, tabla con paginación) se reutilizan en POS, cotizaciones e inventario.

14. Estructura recomendada de base de datos

Base de datos única PostgreSQL con las siguientes convenciones obligatorias:

- Nombres de tablas y campos en inglés, en snake_case, tablas en plural (products, sale_items).
- Clave primaria id de tipo UUID (generada por la aplicación o gen_random_uuid()).
- Campos de trazabilidad: created_at, updated_at (TIMESTAMPTZ) y deleted_at para baja lógica donde aplique.
- Importes en NUMERIC(12,2); cantidades en NUMERIC(12,3) para soportar unidades decimales.
- Estados como VARCHAR con restricción CHECK (o enums de Prisma), nunca texto libre.
- Restricciones en base de datos además de en aplicación: UNIQUE (sku, correlativos, documento de cliente), CHECK (montos ≥ 0 , stock ≥ 0 , debe = haber) y claves foráneas con ON DELETE RESTRICT por defecto.
- Índices: por claves foráneas de alto uso (sale_id, product_id, customer_id), por fechas de consulta de reportes (sale_date, payment_date, created_at) y por estados.
- JSONB solo para metadata de auditoría; las especificaciones de producto usan tabla propia para poder listarse y ordenarse.
- El stock vive en products.stock_current como consulta rápida, pero solo lo modifican los casos de uso de inventario dentro de transacción.

15. Modelo entidad-relación inicial

El modelo se organiza en ocho áreas. Las relaciones principales son:

- Seguridad: roles 1—N users. Opcional: roles N—M permissions vía role_permissions.
- Catálogo: categories 1—N products; units 1—N products; categories 1—N categories (jerarquía por parent_id); products 1—N product_specifications; products 1—N product_images.
- Inventario: products 1—N inventory_movements; users 1—N inventory_movements (created_by); sales 1—N inventory_movements vía reference_type/reference_id.
- Clientes: customers 1—N quotes; customers 1—N sales.
- Cotizaciones: quotes 1—N quote_items; products 1—N quote_items (referencia + snapshot); quotes 0..1—1 sales (converted_sale_id).
- Ventas: sales 1—N sale_items; products 1—N sale_items (referencia + snapshot); users 1—N sales (vendedor).
- Pagos: sales 1—N payments; users 1—N payments (created_by).
- Contabilidad: accounting_entries 1—N accounting_entry_lines; chart_of_accounts 1—N accounting_entry_lines; accounting_entries 0..1—1 accounting_entries (reversed_entry_id).
- Configuración y auditoría: company_settings (singleton), document_sequences (una fila por tipo de documento), users 1—N audit_logs.

16. Diagrama ERD en Mermaid

```

erDiagram
    ROLES ||--o{ USERS : "role_id"
    CATEGORIES ||--o{ CATEGORIES : "parent_id"
    CATEGORIES ||--o{ PRODUCTS : "category_id"
    UNITS ||--o{ PRODUCTS : "unit_id"
    PRODUCTS ||--o{ PRODUCT_SPECIFICATIONS : "product_id"
    PRODUCTS ||--o{ PRODUCT_IMAGES : "product_id"
    PRODUCTS ||--o{ INVENTORY_MOVEMENTS : "product_id"
    USERS ||--o{ INVENTORY_MOVEMENTS : "created_by"
    CUSTOMERS ||--o{ QUOTES : "customer_id"
    USERS ||--o{ QUOTES : "user_id"
    QUOTES ||--o{ QUOTE_ITEMS : "quote_id"
    PRODUCTS ||--o{ QUOTE_ITEMS : "product_id"
    QUOTES ||--o{ SALES : "converted_sale_id"
    CUSTOMERS ||--o{ SALES : "customer_id"
    USERS ||--o{ SALES : "user_id"
    SALES ||--o{ SALE_ITEMS : "sale_id"
    PRODUCTS ||--o{ SALE_ITEMS : "product_id"
    SALES ||--o{ PAYMENTS : "sale_id"
    USERS ||--o{ PAYMENTS : "created_by"
    ACCOUNTING_ENTRIES ||--o{ ACCOUNTING_ENTRY_LINES : "entry_id"
    CHART_OF_ACCOUNTS ||--o{ ACCOUNTING_ENTRY_LINES : "account_id"
    ACCOUNTING_ENTRIES ||--o{ ACCOUNTING_ENTRIES : "reversed_entry_id"
    USERS ||--o{ AUDIT_LOGS : "user_id"

    USERS { uuid id PK varchar username varchar status boolean must_change_password }
    PRODUCTS { uuid id PK varchar sku varchar product_type boolean is_inventory_tracked
numeric stock_current }
    CUSTOMERS { uuid id PK varchar customer_type varchar document_number varchar
customer_stage boolean is_generic }
    QUOTES { uuid id PK varchar quote_number varchar status numeric total }
    SALES { uuid id PK varchar sale_number varchar payment_status varchar sale_status numeric
balance_due }
    PAYMENTS { uuid id PK varchar payment_number varchar payment_method numeric amount
varchar status }
    ACCOUNTING_ENTRIES { uuid id PK varchar entry_number varchar source_type numeric
total_debit numeric total_credit }
    INVENTORY_MOVEMENTS { uuid id PK varchar movement_type numeric quantity numeric
previous_stock numeric new_stock }

```

17. Descripción de entidades principales

Entidad	Área	Rol en el sistema
users	Seguridad	Usuarios internos del sistema con estado y rol asignado.
roles	Seguridad	Roles del sistema precargados por seed (ADMIN, SELLER, WAREHOUSE, MANAGEMENT).
permissions / role_permissions (opcional)	Seguridad	Estructura opcional para permisos granulares. En MVP la autorización se resuelve por rol; estas tablas quedan preparadas para evolución.
categories	Catálogo	Categorías jerárquicas del catálogo (una sola tabla con parent_id).
units	Catálogo	Unidades de medida con control de decimales.
products	Catálogo	Catálogo de productos técnicos, repuestos, equipos y servicios.
product_specifications	Catálogo	Especificaciones técnicas flexibles por producto (clave/valor).
product_images	Catálogo	Imágenes por producto. Almacenamiento local en demo, S3 compatible en producción.
inventory_movements	Inventario	Movimientos trazables de stock. Única vía de modificación del stock.
customers	Clientes	Clientes y prospectos (personas y empresas) en una sola tabla.
quotes	Cotizaciones	Cotizaciones comerciales. No descuentan stock.
quote_items	Cotizaciones	Ítems de cotización con snapshot del producto.
sales	Ventas	Ventas internas (directas o desde cotización). Generan nota de venta interna no fiscal.
sale_items	Ventas	Ítems de venta con snapshot del producto.
payments	Pagos	Pagos asociados a ventas. No existen pagos sueltos.
chart_of_accounts	Contabilidad	Plan de cuentas básico precargado (Caja, Bancos, Cuentas por cobrar, IGV por pagar, Ventas, Descuentos).
accounting_entries	Contabilidad	Asientos precontables automáticos generados por ventas, pagos y anulaciones.
accounting_entry_lines	Contabilidad	Detalle debe/haber por cuenta de cada asiento.
company_settings	Configuración	Configuración de empresa: datos generales, moneda principal, IGV y parámetros comerciales.
document_sequences	Configuración	Correlativos internos por tipo de documento. Se actualizan con bloqueo dentro de transacción.
audit_logs	Auditoría	Registro de solo lectura de acciones críticas con metadata JSON.

18. Tablas sugeridas: campos, tipos, claves y restricciones

Se listan las tablas del MVP con campos, tipos PostgreSQL, claves y restricciones. Los tipos son orientativos y se materializan en schema.prisma.

Tabla users

Usuarios internos del sistema con estado y rol asignado.

Campo	Tipo	Clave / Restricción	Descripción
id	UUID	PK	Identificador único.
username	VARCHAR(50)	UNIQUE, NOT NULL	Usuario de acceso.
email	VARCHAR(150)	UNIQUE, NOT NULL	Correo del usuario.
full_name	VARCHAR(150)	NOT NULL	Nombre completo.
password_hash	VARCHAR(255)	NOT NULL	Hash bcrypt/Argon2id. Nunca texto plano.

Campo	Tipo	Clave / Restricción	Descripción
role_id	UUID	FK → roles.id, NOT NULL	Rol asignado.
status	VARCHAR(20)	CHECK IN ('ACTIVE','INACTIVE','BLOCKED')	Estado del usuario.
must_change_password	BOOLEAN	DEFAULT false	Fuerza cambio en próximo login.
failed_login_attempts	INT	DEFAULT 0	Contador de intentos fallidos.
last_login_at	TIMESTAMPZ	NULL	Último acceso.
created_at / updated_at	TIMESTAMPZ	DEFAULT now()	Trazabilidad temporal.
deleted_at	TIMESTAMPZ	NULL	Baja lógica.

Tabla roles

Roles del sistema precargados por seed (ADMIN, SELLER, WAREHOUSE, MANAGEMENT).

Campo	Tipo	Clave / Restricción	Descripción
id	UUID	PK	Identificador.
code	VARCHAR(30)	UNIQUE, NOT NULL	Código del rol.
name	VARCHAR(80)	NOT NULL	Nombre visible.
description	VARCHAR(255)	NULL	Responsabilidad principal.
created_at / updated_at	TIMESTAMPZ	DEFAULT now()	Trazabilidad.

Tabla permissions / role_permissions (opcional)

Estructura opcional para permisos granulares. En MVP la autorización se resuelve por rol; estas tablas quedan preparadas para evolución.

Campo	Tipo	Clave / Restricción	Descripción
permissions.id	UUID	PK	Identificador de permiso.
permissions.code	VARCHAR(60)	UNIQUE	Código, p. ej. SALES_CANCEL.
role_permissions.role_id	UUID	PK compuesta, FK → roles.id	Rol.
role_permissions.permission_id	UUID	PK compuesta, FK → permissions.id	Permiso.

Tabla categories

Categorías jerárquicas del catálogo (una sola tabla con parent_id).

Campo	Tipo	Clave / Restricción	Descripción
id	UUID	PK	Identificador.
code	VARCHAR(30)	UNIQUE, NOT NULL	Código interno.
name	VARCHAR(100)	NOT NULL, único por nivel (índice parcial)	Nombre.
description	VARCHAR(255)	NULL	Descripción.
parent_id	UUID	FK → categories.id, NULL	Categoría padre (jerarquía). Sin ciclos.

Campo	Tipo	Clave / Restricción	Descripción
status	VARCHAR(20)	CHECK IN ('ACTIVE','INACTIVE')	Estado.
sort_order	INT	DEFAULT 0	Orden de presentación.
created_at / updated_at / deleted_at	TIMESTAMPZ	—	Trazabilidad y baja lógica.

Tabla units

Unidades de medida con control de decimales.

Campo	Tipo	Clave / Restricción	Descripción
id	UUID	PK	Identificador.
code	VARCHAR(15)	UNIQUE, NOT NULL	Código (UND, MTR, KG...).
name	VARCHAR(80)	NOT NULL	Nombre.
abbreviation	VARCHAR(10)	NOT NULL	Abreviatura.
allow_decimal	BOOLEAN	DEFAULT false	Si false, cantidades enteras obligatorias.
status	VARCHAR(20)	CHECK IN ('ACTIVE','INACTIVE')	Estado.
created_at / updated_at / deleted_at	TIMESTAMPZ	—	Trazabilidad y baja lógica.

Tabla products

Catálogo de productos técnicos, repuestos, equipos y servicios.

Campo	Tipo	Clave / Restricción	Descripción
id	UUID	PK	Identificador.
sku	VARCHAR(40)	UNIQUE, NOT NULL	Código único de producto.
name	VARCHAR(180)	NOT NULL	Nombre comercial.
brand	VARCHAR(80)	NULL	Marca.
product_type	VARCHAR(20)	CHECK IN ('PRODUCT','SERVICE')	Tipo de ítem.
category_id	UUID	FK → categories.id, NOT NULL	Categoría.
unit_id	UUID	FK → units.id, NOT NULL	Unidad de medida.
sale_price	NUMERIC(12,2)	NOT NULL, CHECK >= 0	Precio de venta.
commercial_description	TEXT	NULL	Descripción comercial (visible en documentos).
internal_notes	TEXT	NULL	Observación interna (no visible al cliente).
is_inventory_tracked	BOOLEAN	DEFAULT true	Si controla stock. Servicios: false.
stock_current	NUMERIC(12,3)	DEFAULT 0, CHECK >= 0	Stock actual (solo se actualiza por movimientos).
stock_minimum	NUMERIC(12,3)	DEFAULT 0	Stock mínimo para alerta.
status	VARCHAR(20)	CHECK IN ('ACTIVE','INACTIVE')	Estado.

Campo	Tipo	Clave / Restricción	Descripción
created_at / updated_at / deleted_at	TIMESTAMPZ	—	Trazabilidad y baja lógica.

Tabla product_specifications

Especificaciones técnicas flexibles por producto (clave/valor).

Campo	Tipo	Clave / Restricción	Descripción
id	UUID	PK	Identificador.
product_id	UUID	FK → products.id, NOT NULL	Producto.
spec_name	VARCHAR(80)	NOT NULL	Nombre del atributo (p. ej. Potencia).
spec_value	VARCHAR(255)	NOT NULL	Valor (p. ej. 3.300 KW).
sort_order	INT	DEFAULT 0	Orden de presentación.

Tabla product_images

Imágenes por producto. Almacenamiento local en demo, S3 compatible en producción.

Campo	Tipo	Clave / Restricción	Descripción
id	UUID	PK	Identificador.
product_id	UUID	FK → products.id, NOT NULL	Producto.
file_path	VARCHAR(255)	NOT NULL	Ruta o clave del archivo.
is_primary	BOOLEAN	DEFAULT false	Imagen principal.
created_at	TIMESTAMPZ	DEFAULT now()	Fecha de carga.

Tabla inventory_movements

Movimientos trazables de stock. Única vía de modificación del stock.

Campo	Tipo	Clave / Restricción	Descripción
id	UUID	PK	Identificador.
product_id	UUID	FK → products.id, NOT NULL	Producto inventariable.
movement_type	VARCHAR(30)	CHECK IN ('INGRESO_MANUAL','AJUSTE_POSITIVO','AJUSTE_NEGATIVO','SALIDA_VENTA','REVERSA_ANULACION')	Tipo de movimiento.
quantity	NUMERIC(12,3)	NOT NULL, CHECK > 0	Cantidad (siempre positiva; el tipo define el signo).
previous_stock	NUMERIC(12,3)	NOT NULL	Stock antes del movimiento.

Campo	Tipo	Clave / Restricción	Descripción
new_stock	NUMERIC(12,3)	NOT NULL, CHECK >= 0	Stock después del movimiento.
reference_type	VARCHAR(20)	NULL	Origen: 'SALE', 'MANUAL'.
reference_id	UUID	NULL	Id del documento o origen (venta).
reason	VARCHAR(255)	NOT NULL en ajustes/manual	Motivo obligatorio en ajustes.
created_by	UUID	FK → users.id, NOT NULL	Usuario que ejecuta.
created_at	TIMESTAMPZ	DEFAULT now()	Fecha del movimiento.

Tabla customers

Clientes y prospectos (personas y empresas) en una sola tabla.

Campo	Tipo	Clave / Restricción	Descripción
id	UUID	PK	Identificador.
customer_type	VARCHAR(10)	CHECK IN ('PERSON','COMPANY')	Tipo de cliente.
document_type	VARCHAR(10)	CHECK IN ('DNI','RUC','CE','OTHER')	Tipo de documento.
document_number	VARCHAR(20)	UNIQUE junto a document_type	N° de documento.
name	VARCHAR(180)	NOT NULL	Nombre o razón social.
trade_name	VARCHAR(180)	NULL	Nombre comercial.
contact_name	VARCHAR(150)	NULL	Contacto (empresas).
phone / email / address	VARCHAR	NULL	Datos de contacto.
customer_stage	VARCHAR(15)	CHECK IN ('PROSPECT','CUSTOMER')	Etapas comerciales.
is_generic	BOOLEAN	DEFAULT false	Marca al cliente "Público general".
status	VARCHAR(20)	CHECK IN ('ACTIVE','INACTIVE','BLOCKED')	Estado.
internal_notes	TEXT	NULL	Notas internas.
created_at / updated_at / deleted_at	TIMESTAMPZ	—	Trazabilidad y baja lógica.

Tabla quotes

Cotizaciones comerciales. No descuentan stock.

Campo	Tipo	Clave / Restricción	Descripción
id	UUID	PK	Identificador.
quote_number	VARCHAR(20)	UNIQUE, NOT NULL	Correlativo COT-XXXXXX generado en backend.
customer_id	UUID	FK → customers.id, NOT NULL	Cliente registrado (no genérico).
user_id	UUID	FK → users.id, NOT NULL	Vendedor que emite.
issue_date	DATE	NOT NULL	Fecha de emisión.
expiration_date	DATE	NOT NULL	Fecha de vencimiento.
subtotal	NUMERIC(12,2)	NOT NULL	Suma de ítems.
discount_amount	NUMERIC(12,2)	DEFAULT 0, CHECK >= 0	Descuento global por monto.
tax_amount	NUMERIC(12,2)	DEFAULT 0	IGV si está activo en configuración.
total	NUMERIC(12,2)	NOT NULL, CHECK >= 0	Total final.
status	VARCHAR(15)	CHECK IN ('PENDIENTE','ACEPTADA','RECHAZADA','VENCIDA','CONVERTIDA')	Estado.
converted_sale_id	UUID	FK → sales.id, NULL	Venta generada al convertir.
notes	VARCHAR(500)	NULL	Observaciones.
created_at / updated_at	TIMESTAMPZ	—	Trazabilidad.

Tabla quote_items

Ítems de cotización con snapshot del producto.

Campo	Tipo	Clave / Restricción	Descripción
id	UUID	PK	Identificador.
quote_id	UUID	FK → quotes.id, NOT NULL, ON DELETE CASCADE	Cotización.
product_id	UUID	FK → products.id, NOT NULL	Producto de referencia.
product_sku / product_name / unit_code	VARCHAR	NOT NULL (snapshot)	Copia histórica de SKU, nombre y unidad.
quantity	NUMERIC(12,3)	NOT NULL, CHECK > 0	Cantidad (entera si la unidad no permite decimales).
unit_price	NUMERIC(12,2)	NOT NULL (snapshot)	Precio al momento de cotizar.

Campo	Tipo	Clave / Restricción	Descripción
line_total	NUMERIC(12,2)	NOT NULL	quantity × unit_price.

Tabla sales

Ventas internas (directas o desde cotización). Generan nota de venta interna no fiscal.

Campo	Tipo	Clave / Restricción	Descripción
id	UUID	PK	Identificador.
sale_number	VARCHAR(20)	UNIQUE, NOT NULL	Correlativo NV-XXXXXX generado en backend.
customer_id	UUID	FK → customers.id, NOT NULL	Cliente (genérico solo si pago completo).
user_id	UUID	FK → users.id, NOT NULL	Vendedor.
quote_id	UUID	FK → quotes.id, NULL	Cotización origen si aplica.
sale_date	TIMESTAMPZ	NOT NULL	Fecha de venta.
subtotal / discount_amount / tax_amount / total	NUMERIC(12,2)	NOT NULL, total >= 0	Importes de la venta.
paid_amount	NUMERIC(12,2)	DEFAULT 0	Suma de pagos ACTIVE.
balance_due	NUMERIC(12,2)	DEFAULT 0, CHECK >= 0	Saldo pendiente (total – paid_amount).
payment_status	VARCHAR(20)	CHECK IN ('UNPAID','PARTIALLY_PAID','PAID')	Estado de pago.
sale_status	VARCHAR(20)	CHECK IN ('CONFIRMED','CANCELLED')	Estado de la venta.
delivery_status	VARCHAR(15)	CHECK IN ('NO_APLICA','PENDIENTE','ENTREGADA','OBSERVADA')	Estado de entrega separado.
cancellation_reason	VARCHAR(255)	NOT NULL si CANCELLED	Motivo obligatorio de anulación.
cancelled_by / cancelled_at	UUID / TIMESTAMPZ	NULL	Trazabilidad de anulación.
created_at / updated_at	TIMESTAMPZ	—	Trazabilidad.

Tabla sale_items

Ítems de venta con snapshot del producto.

Campo	Tipo	Clave / Restricción	Descripción
id	UUID	PK	Identificador.
sale_id	UUID	FK → sales.id, NOT NULL, ON DELETE CASCADE	Venta.
product_id	UUID	FK → products.id, NOT NULL	Producto de referencia.

Campo	Tipo	Clave / Restricción	Descripción
product_sku / product_name / unit_code	VARCHAR	NOT NULL (snapshot)	Copia histórica.
is_inventory_tracked	BOOLEAN	NOT NULL (snapshot)	Si descontó stock.
quantity	NUMERIC(12,3)	NOT NULL, CHECK > 0	Cantidad.
unit_price	NUMERIC(12,2)	NOT NULL (snapshot)	Precio aplicado.
line_total	NUMERIC(12,2)	NOT NULL	quantity × unit_price.

Tabla payments

Pagos asociados a ventas. No existen pagos sueltos.

Campo	Tipo	Clave / Restricción	Descripción
id	UUID	PK	Identificador.
payment_number	VARCHAR(20)	UNIQUE, NOT NULL	Correlativo PAG-XXXXXX.
sale_id	UUID	FK → sales.id, NOT NULL	Venta asociada.
payment_method	VARCHAR(20)	CHECK IN ('CASH', 'BANK_TRANSFER', 'BANK_DEPOSIT', 'CARD', 'DIGITAL_WALLET', 'OTHER')	Método de pago.
amount	NUMERIC(12,2)	NOT NULL, CHECK > 0	Monto (≤ saldo pendiente).
reference_number	VARCHAR(60)	NOT NULL para transferencia/depósito/tarjeta	N° de operación.
payment_date	TIMESTAMP TZ	NOT NULL	Fecha del cobro.
status	VARCHAR(15)	CHECK IN ('ACTIVE', 'CANCELLED')	Estado del pago.
cancellation_reason	VARCHAR(255)	NOT NULL si CANCELLED	Motivo obligatorio.
created_by	UUID	FK → users.id, NOT NULL	Usuario que registra.
created_at	TIMESTAMP TZ	DEFAULT now()	Registro.

Tabla chart_of_accounts

Plan de cuentas básico precargado (Caja, Bancos, Cuentas por cobrar, IGV por pagar, Ventas, Descuentos).

Campo	Tipo	Clave / Restricción	Descripción
id	UUID	PK	Identificador.
code	VARCHAR(15)	UNIQUE, NOT NULL	Código de cuenta (p. ej. 101).
name	VARCHAR(120)	NOT NULL	Nombre de la cuenta.
account_type	VARCHAR(20)	CHECK IN ('ASSET', 'LIABILITY', 'INCOME', 'EXPENSE')	Naturaleza.

Campo	Tipo	Clave / Restricción	Descripción
status	VARCHAR(20)	CHECK IN ('ACTIVE','INACTIVE')	Estado.

Tabla accounting_entries

Asientos precontables automáticos generados por ventas, pagos y anulaciones.

Campo	Tipo	Clave / Restricción	Descripción
id	UUID	PK	Identificador.
entry_number	VARCHAR(20)	UNIQUE, NOT NULL	Correlativo AST-XXXXXX.
entry_date	TIMESTAMPTZ	NOT NULL	Fecha del asiento.
source_type	VARCHAR(20)	CHECK IN ('SALE','PAYMENT','SALE_CANCELLATION','PAYMENT_CANCELLATION')	Operación origen.
source_id	UUID	NOT NULL	Id de venta o pago origen.
description	VARCHAR(255)	NOT NULL	Glosa.
total_debit / total_credit	NUMERIC(12,2)	NOT NULL, CHECK total_debit = total_credit	Debe y haber cuadrados.
reversed_entry_id	UUID	FK → accounting_entries.id, NULL	Asiento que revierte (trazabilidad).
is_reversed	BOOLEAN	DEFAULT false	Marcado al ser revertido.
created_at	TIMESTAMPTZ	DEFAULT now()	Registro. No se elimina físicamente.

Tabla accounting_entry_lines

Detalle debe/haber por cuenta de cada asiento.

Campo	Tipo	Clave / Restricción	Descripción
id	UUID	PK	Identificador.
entry_id	UUID	FK → accounting_entries.id, NOT NULL, ON DELETE RESTRICT	Asiento.
account_id	UUID	FK → chart_of_accounts.id, NOT NULL	Cuenta contable.
debit	NUMERIC(12,2)	DEFAULT 0, CHECK >= 0	Importe al debe.
credit	NUMERIC(12,2)	DEFAULT 0, CHECK >= 0	Importe al haber.

Tabla company_settings

Configuración de empresa: datos generales, moneda principal, IGV y parámetros comerciales.

Campo	Tipo	Clave / Restricción	Descripción
id	UUID	PK	Registro único (singleton).

Campo	Tipo	Clave / Restricción	Descripción
company_name / trade_name	VARCHAR(180)	NOT NULL	Razón social y nombre comercial de demo.
tax_id	VARCHAR(20)	NULL	RUC de demo (no fiscal).
address / phone / email	VARCHAR	NULL	Datos de contacto.
logo_path	VARCHAR(255)	NULL	Logo para documentos.
currency_code	VARCHAR(5)	DEFAULT 'PEN'	Moneda principal única del MVP.
currency_symbol	VARCHAR(5)	DEFAULT 'S/'	Símbolo.
tax_enabled	BOOLEAN	DEFAULT true	IGV activado/desactivado (configurable).
tax_rate	NUMERIC(5,2)	DEFAULT 18.00	Tasa de IGV.
quote_validity_days	INT	DEFAULT 15	Vigencia por defecto de cotizaciones.
max_discount_percent	NUMERIC(5,2)	DEFAULT 100	Límite de descuento configurable.
updated_at / updated_by	TIMESTAMPZ / UUID	—	Trazabilidad de cambios (auditados).

Tabla document_sequences

Correlativos internos por tipo de documento. Se actualizan con bloqueo dentro de transacción.

Campo	Tipo	Clave / Restricción	Descripción
id	UUID	PK	Identificador.
document_type	VARCHAR(20)	UNIQUE, CHECK IN ('QUOTE', 'SALE', 'PAYMENT', 'ACCOUNTING_ENTRY')	Tipo de documento.
prefix	VARCHAR(10)	NOT NULL	Prefijo: COT, NV, PAG, AST.
current_number	BIGINT	DEFAULT 0	Último número emitido.
padding	INT	DEFAULT 6	Ceros a la izquierda.
updated_at	TIMESTAMPZ	—	Última emisión.

Tabla audit_logs

Registro de solo lectura de acciones críticas con metadata JSON.

Campo	Tipo	Clave / Restricción	Descripción
id	UUID	PK	Identificador.
user_id	UUID	FK → users.id, NULL en login fallido	Usuario ejecutor.
module	VARCHAR(40)	NOT NULL	Módulo (AUTH, SALES, INVENTORY...).
action	VARCHAR(60)	NOT NULL	Acción (LOGIN_SUCCESS, SALE_CANCELLED...).
entity_type	VARCHAR(40)	NULL	Entidad afectada.
entity_id	UUID	NULL	Id de la entidad.
description	VARCHAR(255)	NULL	Descripción legible.
metadata	JSONB	NULL	Detalle old/new y datos de contexto.
ip_address	VARCHAR(45)	NULL	IP de origen.

Campo	Tipo	Clave / Restricción	Descripción
created_at	TIMESTAMPZ	DEFAULT now()	Fecha. Registro inmutable.

19. Relaciones entre tablas

Tabla origen	Campo FK	Tabla destino	Cardinalidad	Regla de borrado
users	role_id	roles	N : 1	RESTRICT
categories	parent_id	categories	N : 1 (jerarquía)	RESTRICT
products	category_id	categories	N : 1	RESTRICT
products	unit_id	units	N : 1	RESTRICT
product_specifications	product_id	products	N : 1	CASCADE
product_images	product_id	products	N : 1	CASCADE
inventory_movements	product_id	products	N : 1	RESTRICT
inventory_movements	created_by	users	N : 1	RESTRICT
quotes	customer_id	customers	N : 1	RESTRICT
quotes	user_id	users	N : 1	RESTRICT
quotes	converted_sale_id	sales	0..1 : 1	SET NULL
quote_items	quote_id	quotes	N : 1	CASCADE
quote_items	product_id	products	N : 1	RESTRICT
sales	customer_id	customers	N : 1	RESTRICT
sales	user_id	users	N : 1	RESTRICT
sales	quote_id	quotes	N : 0..1	SET NULL
sale_items	sale_id	sales	N : 1	CASCADE
sale_items	product_id	products	N : 1	RESTRICT
payments	sale_id	sales	N : 1	RESTRICT
payments	created_by	users	N : 1	RESTRICT
accounting_entry_lines	entry_id	accounting_entries	N : 1	RESTRICT
accounting_entry_lines	account_id	chart_of_accounts	N : 1	RESTRICT
accounting_entries	reversed_entry_id	accounting_entries	0..1 : 1	RESTRICT
audit_logs	user_id	users	N : 1	SET NULL

Nota: la baja lógica (deleted_at / status) es el mecanismo estándar para registros con historial; el borrado físico solo aplica a ítems hijos vía CASCADE cuando se elimina un borrador sin efectos (caso excepcional).

20. Flujos técnicos por módulo

Se describen los flujos técnicos principales tal como deben implementarse en los casos de uso. Todos los flujos que afectan varias tablas se ejecutan dentro de una transacción Prisma (\$transaction).

20.1 Autenticación

- Login: el controller recibe credenciales → el servicio busca usuario por username/email → valida estado ACTIVE → verifica hash → resetea failed_login_attempts → emite JWT y lo escribe en cookie HttpOnly + Secure + SameSite → audita LOGIN_SUCCESS.

- Login fallido: respuesta 401 con mensaje genérico → incrementa `failed_login_attempts` → si supera el máximo configurado, cambia estado a `BLOCKED` → audita `LOGIN_FAILED`.
- Reset de contraseña: solo ADMIN → actualiza `password_hash` con contraseña temporal → marca `must_change_password = true` → audita. En el siguiente login el frontend fuerza el cambio.

20.2 Catálogo (categorías, unidades, productos)

- Crear producto: valida SKU único → valida categoría y unidad `ACTIVE` → si `is_inventory_tracked`, exige `stock_minimum` → persiste producto + especificaciones + imágenes → audita `PRODUCT_CREATED`.
- Editar producto: el stock nunca se modifica desde esta pantalla; cambios de precio o estado se auditan con metadata `old/new`.
- Inactivar categoría/unidad/producto con historial: baja lógica (status `INACTIVE` o `deleted_at`); nunca `DELETE` físico.

20.3 Inventario

- Registrar movimiento manual (`INGRESO_MANUAL`, `AJUSTE_POSITIVO`, `AJUSTE_NEGATIVO`): transacción → `SELECT ... FOR UPDATE` del producto → valida cantidad > 0 y motivo obligatorio → valida stock suficiente si es negativo → calcula `previous_stock` y `new_stock` → actualiza `products.stock_current` → inserta `inventory_movements` → audita.
- Salida por venta (`SALIDA_VENTA`) y reversa por anulación (`REVERSA_ANULACION`): las ejecuta `InventoryService` dentro de la transacción de venta/anulación, con `reference_type = SALE` y `reference_id = sale.id`.

20.4 Clientes

- Registrar cliente: valida duplicidad por (`document_type`, `document_number`) → guarda con `customer_stage` y status `ACTIVE` → audita.
- Registro rápido desde POS: formulario reducido (tipo, documento, nombre, teléfono) que reutiliza el mismo endpoint de creación.
- Selección en venta: si la venta quedará con saldo pendiente, el sistema bloquea la selección del cliente genérico "Público general" y de clientes `BLOCKED`.

20.5 Cotizaciones

- Crear cotización: valida cliente registrado (no genérico) y al menos un ítem → toma snapshot de SKU, nombre, unidad y precio → valida descuento sin total negativo → genera correlativo `COT` dentro de la transacción → estado `PENDIENTE` → audita. El stock mostrado es referencial (no se descuenta ni se reserva).
- Convertir a venta: valida estado convertible (no `RECHAZADA`, `VENCIDA` ni `CONVERTIDA` y dentro de vigencia) → delega en el caso de uso `create-sale` con `quote_id` → si la venta se confirma, marca la cotización `CONVERTIDA` y guarda `converted_sale_id`, todo en la misma transacción.
- Vencimiento: al consultar/listar, si `expiration_date < hoy` y estado `PENDIENTE`, se presenta y/o actualiza como `VENCIDA` (job diario opcional).

20.6 Ventas / POS

- Crear venta (directa o desde cotización) — transacción única: (1) valida cliente y reglas de crédito, (2) valida ítems: producto `ACTIVE`, cantidad válida según `allow_decimal`, (3) bloquea productos inventariables (`FOR UPDATE`) y valida stock suficiente, (4) genera correlativo `NV`, (5) inserta `sales + sale_items` con snapshot, (6) descuenta stock e inserta movimientos `SALIDA_VENTA`, (7) registra pago inicial si se envía, (8) genera asiento contable `SALE` (y `PAYMENT` si hubo pago), (9) calcula `payment_status`, (10) audita `SALE_CREATED`. Si cualquier paso falla, rollback total.
- Anular venta — transacción única: solo ADMIN → valida que no esté `ANULADA` → exige motivo → marca `sale_status CANCELLED` → reversa stock (`REVERSA_ANULACION`) → marca pagos según regla y genera reversión contable → audita `SALE_CANCELLED`.

20.7 Pagos

- Registrar pago: transacción → bloquea la venta → valida no ANULADA, monto > 0 y $\leq$ balance_due, método válido y referencia obligatoria si aplica → inserta payment ACTIVE con correlativo PAG → recalcula paid_amount, balance_due y payment_status → genera asiento PAYMENT → audita.
- Anular pago: solo ADMIN → motivo obligatorio → marca CANCELLED → recalcula saldo y estado de la venta → genera asiento de reversión PAYMENT_CANCELLATION → audita.

20.8 Contabilidad básica

- AccountingService expone createEntryFromSale, createEntryFromPayment y createReversalEntry; siempre se invocan dentro de la transacción de la operación origen.
- Cada asiento valida internamente total_debit = total_credit antes de persistir; si no cuadra, lanza excepción y aborta toda la transacción.

20.9 Configuración y correlativos

- Actualizar configuración: solo ADMIN → valida datos → guarda → audita con metadata old/new.
- Generar correlativo: dentro de la transacción del documento → UPDATE document_sequences SET current_number = current_number + 1 WHERE document_type = \$1 RETURNING ... (bloqueo de fila) → formatea prefix + número con padding → asigna al documento. El frontend jamás genera números.

20.10 Auditoría

- AuditService.log({ userId, module, action, entityType, entityId, description, metadata, ip }) se invoca desde los casos de uso. Para acciones críticas se registra dentro de la transacción; el registro es de solo lectura (sin endpoints de edición/eliminación).

21. Diagramas de flujo Mermaid para procesos principales

21.1 Venta directa (POS)

```

flowchart TD
    A[Vendedor abre POS] --> B[Selecciona cliente o Publico general]
    B --> C[Agrega productos al carrito]
    C --> D{Stock suficiente en inventariables?}
    D -- No --> E[Bloquea confirmacion y muestra stock disponible]
    D -- Si --> F[Calcula subtotal, descuento, IGV y total]
    F --> G{Venta quedara con saldo pendiente?}
    G -- "Si y cliente generico" --> H[Exige cliente registrado]
    G -- No / cliente registrado --> I[Confirmar venta - inicia transaccion]
    I --> J[Genera correlativo NV]
    J --> K[Inserta sales + sale_items con snapshot]
    K --> L[Descuenta stock + movimientos SALIDA_VENTA]
    L --> M{Registra pago inicial?}
    M -- Si --> N[Inserta payment + asiento PAYMENT]
    M -- No --> O[payment_status = UNPAID]
    N --> P[Asiento SALE + auditoria]
    O --> P
    P --> Q[Commit y nota de venta imprimible]
  
```

21.2 Conversión de cotización a venta

```

flowchart TD
    A[Vendedor abre cotizacion] --> B{Estado convertible?}
    B -- "RECHAZADA / VENCIDA / CONVERTIDA" --> C[Bloquear conversion]
    B -- "PENDIENTE o ACEPTADA vigente" --> D[Copia cliente e items]
    D --> E{Stock actual suficiente?}
    E -- No --> F[Muestra faltantes y detiene]
    E -- Si --> G[Crea venta con flujo estandar en transaccion]
    G --> H[Marca cotizacion CONVERTIDA + converted_sale_id]
    H --> I[Commit + auditoria]
  
```

21.3 Registro de pago posterior

```

flowchart TD
    A[Usuario abre venta con saldo] --> B{Venta anulada?}
    B -- Si --> C[Rechazar pago]
    B -- No --> D[Ingresa metodo, monto y referencia]
    D --> E{0 < monto <= balance_due?}
    E -- No --> F[Error de validacion]
    E -- Si --> G{Metodo requiere referencia?}
    G -- "Si y falta" --> F
    G -- OK --> H[Transaccion: inserta payment ACTIVE]
    H --> I[Recalcula paid_amount, balance_due, payment_status]
    I --> J[Asiento PAYMENT: Debe Caja/Bancos, Haber CxC]
    J --> K[Auditoria + commit]

```

21.4 Anulación de venta

```

flowchart TD
    A[ADMIN abre venta] --> B{Ya anulada?}
    B -- Si --> C[Rechazar]
    B -- No --> D[Ingresa motivo obligatorio]
    D --> E[Transaccion: sale_status = CANCELLED]
    E --> F[Reversa stock: movimientos REVERSA_ANULACION]
    F --> G[Marca pagos segun regla de reversion]
    G --> H[Asiento de reversion vinculado reversed_entry_id]
    H --> I[Auditoria SALE_CANCELLED + commit]

```

22. Reglas de negocio por módulo

Autenticación y usuarios

- Solo usuarios ACTIVE inician sesión; INACTIVE y BLOCKED son rechazados.
- Solo ADMIN crea, edita, bloquea/desbloquea usuarios y resetea contraseñas.
- No se puede bloquear ni inactivar al único administrador activo.
- Mensajes de error de login genéricos (sin revelar si el usuario existe).
- must_change_password = true fuerza cambio de contraseña antes de operar.

Categorías y unidades

- Nombre de categoría único dentro del mismo nivel (mismo parent_id).
- Prohibidos los ciclos padre/hija en categorías.
- Categoría o unidad inactiva no aparece al crear nuevos productos.
- Si la unidad tiene allow_decimal = false, las cantidades de venta, cotización e inventario deben ser enteras.
- Con productos asociados no se eliminan físicamente; se inactivan.

Productos

- SKU único en todo el catálogo.
- Producto INACTIVE no puede agregarse a nuevas ventas ni cotizaciones.
- Producto inventariable maneja stock_current y stock_minimum; el stock solo cambia por movimientos de inventario.
- Servicios (product_type = SERVICE, is_inventory_tracked = false) no descuentan stock.
- Cotizaciones y ventas guardan snapshot de nombre, SKU, unidad y precio.
- Productos con historial no se eliminan físicamente.

Inventario

- Solo productos inventariables generan movimientos.
- Cantidad de movimiento siempre mayor a cero.

- No se permite stock negativo: toda salida valida stock disponible.
- Ajustes manuales exigen motivo obligatorio.
- Cada movimiento persiste `previous_stock` y `new_stock`.
- Stock bajo: `stock_current` $\leq$ `stock_minimum` (solo inventariables).

Cientes

- No duplicar clientes por (`document_type`, `document_number`).
- Cliente genérico "Público general" solo para ventas al contado (pago completo).
- Cotizaciones exigen cliente registrado; no se cotiza a Público general.
- Cliente BLOCKED no puede generar ventas con saldo pendiente.
- Cliente inactivo no aparece por defecto en operaciones nuevas.
- Clientes con historial no se eliminan físicamente.

Cotizaciones

- Toda cotización requiere cliente registrado y al menos un ítem.
- Cotización no descuenta ni reserva stock; el stock se valida informativamente al cotizar y de forma obligatoria al convertir.
- Cotización CONVERTIDA no se edita; RECHAZADA o VENCIDA no se convierte directamente.
- Descuento global por monto no puede generar total negativo.
- El correlativo COT es único y lo genera el backend.

Ventas / POS

- Venta con al menos un ítem; no se venden productos inactivos.
- No se vende inventariable sin stock suficiente; el descuento de stock ocurre al confirmar, en transacción.
- Venta confirmada no se edita: si hay error, se anula y se crea otra.
- Venta con saldo pendiente exige cliente registrado (no Público general).
- Solo ADMIN anula ventas, con motivo obligatorio.
- Venta ANULADA no recibe pagos y revierte stock, pagos y contabilidad.
- El correlativo NV es único y lo genera el backend.

Pagos

- Todo pago pertenece a una venta; no existen pagos sueltos.
- No se paga una venta anulada.
- Monto mayor a cero y no mayor al saldo pendiente.
- Referencia obligatoria para BANK_TRANSFER, BANK_DEPOSIT y CARD.
- Pagos no se editan: se anulan con motivo y se registra uno nuevo.
- Pagos CANCELLED no suman en cobros ni en `paid_amount`.

Contabilidad básica

- Toda venta confirmada genera asiento SALE (Debe: Cuentas por cobrar; Haber: Ventas e IGV si activo).
- Todo pago genera asiento PAYMENT (Debe: Caja/Bancos según método; Haber: Cuentas por cobrar).
- Toda anulación genera asiento inverso vinculado por `reversed_entry_id`.
- Debe y haber deben cuadrar; los asientos no se eliminan físicamente.
- No hay asientos manuales en MVP; solo ADMIN y MANAGEMENT consultan.
- El módulo no genera libros oficiales ni reemplaza al sistema contable tributario.

Reportes y dashboard

- Ventas y pagos anulados se excluyen de los totales principales.
- Productos no inventariables no aparecen en stock bajo.
- Los datos se filtran en backend según el rol del usuario.

Configuración y correlativos

- Solo ADMIN edita configuración; los cambios se auditan con old/new.
- Una moneda principal; IGV configurable sin implicar facturación electrónica.
- Correlativos únicos, generados en backend dentro de transacciones; el frontend nunca los genera.
- La nota de venta interna no es comprobante fiscal.
- Vigencia de cotización y límite de descuento son configurables.

Auditoría

- Se auditan acciones críticas (login, anulaciones, ajustes, cambios de configuración y precio), no búsquedas ni listados.
- La auditoría es de solo lectura: sin edición ni eliminación desde interfaz.
- Solo ADMIN consulta auditoría completa; MANAGEMENT opcional.

23. Validaciones funcionales y técnicas

Todas las validaciones se ejecutan en backend (DTOs con class-validator + reglas de negocio en casos de uso + restricciones en base de datos). El frontend replica las validaciones solo para mejorar la experiencia.

Ámbito	Validación	Capa que la garantiza
Entrada HTTP	Tipos, formatos, longitudes, campos requeridos y enums en todos los endpoints (ValidationPipe global con whitelist).	DTO (class-validator)
Unicidad	sku, username, email, correlativos, (document_type, document_number) de clientes, code de categorías/unidades.	Índice UNIQUE + manejo de error P2002
Cantidades	quantity > 0; entera si la unidad tiene allow_decimal = false.	Caso de uso + CHECK
Montos	Precios y montos ≥ 0; pago > 0 y ≤ balance_due; descuento sin total negativo.	Caso de uso + CHECK
Stock	new_stock ≥ 0 en toda salida o ajuste negativo; bloqueo de fila para evitar carreras.	Caso de uso + CHECK + FOR UPDATE
Estados	Transiciones válidas: cotización convertible, venta no anulada para pagar, pago ACTIVE para anular.	Caso de uso
Referencias de pago	reference_number obligatorio para transferencia, depósito y tarjeta.	DTO condicional + caso de uso
Contabilidad	total_debit = total_credit en cada asiento.	Servicio + CHECK
Autorización	Rol permitido por endpoint (RolesGuard); nunca solo ocultar botones en frontend.	Guard backend
Fechas	expiration_date ≥ issue_date en cotizaciones; rangos válidos en filtros de reportes.	DTO + caso de uso

24. API endpoints sugeridos por módulo

API REST bajo el prefijo /api/v1. Todos los endpoints (salvo login) requieren JWT válido; la columna Roles indica la autorización mínima. Los listados soportan paginación (?page=&pageSize=) y filtros propios.

Autenticación

Método	Ruta	Descripción	Roles
POST	/auth/login	Iniciar sesión; setea cookie HttpOnly.	Público
POST	/auth/logout	Cerrar sesión; limpia cookie.	Autenticado
GET	/auth/me	Perfil y rol del usuario autenticado.	Autenticado
POST	/auth/change-password	Cambio de contraseña propio (incluye must_change_password).	Autenticado

Usuarios

Método	Ruta	Descripción	Roles
GET	/users	Listar usuarios con filtros.	ADMIN
POST	/users	Crear usuario.	ADMIN
PATCH	/users/:id	Editar usuario (rol, estado, datos).	ADMIN
POST	/users/:id/reset-password	Reset con contraseña temporal.	ADMIN
POST	/users/:id/block /unblock	Bloquear / desbloquear.	ADMIN

Categorías y unidades

Método	Ruta	Descripción	Roles
GET/POST	/categories · /units	Listar árbol / crear.	ADMIN (crear), resto consulta
PATCH	/categories/:id · /units/:id	Editar o inactivar.	ADMIN

Productos

Método	Ruta	Descripción	Roles
GET	/products	Listado con filtros: texto, categoría, marca, estado, stock bajo.	Todos los roles
GET	/products/:id	Detalle con especificaciones e imágenes.	Todos los roles
POST	/products	Crear producto.	ADMIN
PATCH	/products/:id	Editar (sin tocar stock).	ADMIN
POST	/products/:id/images	Subir imagen.	ADMIN

Inventario

Método	Ruta	Descripción	Roles
GET	/inventory/stock	Stock actual con filtros y stock bajo.	ADMIN, WAREHOUSE, MANAGEMENT
GET	/inventory/movements	Historial de movimientos con filtros.	ADMIN, WAREHOUSE, MANAGEMENT
POST	/inventory/movements	Registrar INGRESO_MANUAL o AJUSTE_*.	ADMIN, WAREHOUSE

Clientes

Método	Ruta	Descripción	Roles
GET	/customers	Búsqueda por documento, nombre o teléfono.	ADMIN, SELLER, MANAGEMENT
POST	/customers	Crear cliente (incluye registro rápido).	ADMIN, SELLER
PATCH	/customers/:id	Editar / inactivar / bloquear.	ADMIN (SELLER edita datos básicos)
GET	/customers/:id/summary	Detalle con ventas, cotizaciones y saldo.	ADMIN, SELLER, MANAGEMENT

Cotizaciones

Método	Ruta	Descripción	Roles
GET	/quotes	Listado con filtros por estado, fechas, cliente.	ADMIN, SELLER, MANAGEMENT
POST	/quotes	Crear cotización.	ADMIN, SELLER
GET	/quotes/:id	Detalle con ítems.	ADMIN, SELLER, MANAGEMENT
PATCH	/quotes/:id/status	Marcar ACEPTADA / RECHAZADA.	ADMIN, SELLER
POST	/quotes/:id/convert	Convertir a venta (transaccional).	ADMIN, SELLER
GET	/quotes/:id/pdf	PDF simple de la cotización.	ADMIN, SELLER, MANAGEMENT

Ventas / POS

Método	Ruta	Descripción	Roles
POST	/sales	Crear venta directa o desde cotización (con pago inicial opcional).	ADMIN, SELLER
GET	/sales	Listado con filtros por fechas, estado, vendedor, cliente.	ADMIN, SELLER, MANAGEMENT
GET	/sales/:id	Detalle: ítems, pagos, movimientos, asientos.	ADMIN, SELLER, MANAGEMENT
POST	/sales/:id/cancel	Anular venta con motivo (transaccional).	ADMIN
GET	/sales/:id/receipt	Nota de venta interna imprimible (PDF).	ADMIN, SELLER

Pagos

Método	Ruta	Descripción	Roles
POST	/payments	Registrar pago sobre una venta.	ADMIN, SELLER
GET	/payments	Listado por fechas, método, estado.	ADMIN, SELLER, MANAGEMENT
POST	/payments/:id/cancel	Anular pago con motivo.	ADMIN
GET	/receivables	Cuentas por cobrar (ventas con saldo).	ADMIN, SELLER, MANAGEMENT

Contabilidad básica

Método	Ruta	Descripción	Roles
GET	/accounting/entries	Libro diario interno con filtros.	ADMIN, MANAGEMENT
GET	/accounting/entries/:id	Detalle de asiento con líneas.	ADMIN, MANAGEMENT
GET	/accounting/accounts	Plan de cuentas básico.	ADMIN, MANAGEMENT

Dashboard y reportes

Método	Ruta	Descripción	Roles
GET	/dashboard/summary	Indicadores según rol.	Todos (contenido por rol)
GET	/reports/sales-by-date	Reporte 1 con filtros.	Según matriz de permisos
GET	/reports/...	Un endpoint por cada uno de los 10 reportes (ver sección 37).	Según matriz de permisos

Configuración

Método	Ruta	Descripción	Roles
GET	/settings/company	Consultar configuración.	ADMIN (MANAGEMENT consulta)
PATCH	/settings/company	Actualizar configuración (auditado).	ADMIN
GET	/settings/sequences	Estado de correlativos.	ADMIN

Auditoría

Método	Ruta	Descripción	Roles
GET	/audit-logs	Consulta con filtros: fecha, usuario, módulo, acción, entidad.	ADMIN (MANAGEMENT opcional)

25. Contratos request/response JSON para endpoints principales

25.1 POST /api/v1/auth/login

```
// Request
{ "usernameOrEmail": "vendedor01", "password": "*****" }

// Response 200 (el JWT viaja en cookie HttpOnly, no en el body)
{
  "user": {
    "id": "8f2c...", "username": "vendedor01",
    "fullName": "Vendedor Demo", "role": "SELLER",
    "mustChangePassword": false
  }
}

// Response 401 (mensaje genérico, no revela existencia del usuario)
{ "statusCode": 401, "errorCode": "AUTH_INVALID_CREDENTIALS",
  "message": "Usuario o contraseña incorrectos." }
```

25.2 POST /api/v1/sales (venta directa con pago completo)

```
// Request
```

```

{
  "customerId": "c1a2...",           // o id del cliente "Público general"
  "quoteId": null,                   // uuid si viene de cotización
  "discountAmount": 50.00,
  "items": [
    { "productId": "p001...", "quantity": 2, "unitPrice": 1500.00 },
    { "productId": "p045...", "quantity": 1, "unitPrice": 250.00 }
  ],
  "initialPayment": {
    "paymentMethod": "CASH", "amount": 3776.00, "referenceNumber": null
  }
}

// Response 201
{
  "id": "s9d8...", "saleNumber": "NV-000124",
  "customer": { "id": "c1a2...", "name": "Constructora San Miguel S.A.C." },
  "subtotal": 3250.00, "discountAmount": 50.00,
  "taxAmount": 576.00, "total": 3776.00,
  "paidAmount": 3776.00, "balanceDue": 0.00,
  "paymentStatus": "PAID", "saleStatus": "CONFIRMED",
  "deliveryStatus": "PENDIENTE",
  "items": [
    { "productSku": "GEN-3300", "productName": "Generador 3.300 KW",
      "quantity": 2, "unitPrice": 1500.00, "lineTotal": 3000.00 }
  ]
}

// Response 422 (stock insuficiente)
{ "statusCode": 422, "errorCode": "INVENTORY_INSUFFICIENT_STOCK",
  "message": "Stock insuficiente para GEN-3300. Disponible: 1.",
  "details": [ { "productId": "p001...", "available": 1, "requested": 2 } ] }

```

25.3 POST /api/v1/payments (pago parcial posterior)

```

// Request
{
  "saleId": "s9d8...",
  "paymentMethod": "BANK_TRANSFER",
  "amount": 1000.00,
  "referenceNumber": "OP-778812",
  "paymentDate": "2026-07-18T15:30:00Z"
}

// Response 201
{
  "id": "pay7...", "paymentNumber": "PAG-000310",
  "saleNumber": "NV-000124", "amount": 1000.00,
  "sale": { "paidAmount": 2000.00, "balanceDue": 1776.00,
    "paymentStatus": "PARTIALLY_PAID" }
}

// Response 422 (excede saldo)
{ "statusCode": 422, "errorCode": "PAYMENT_EXCEEDS_BALANCE",
  "message": "El monto supera el saldo pendiente (1776.00)." }

```

25.4 POST /api/v1/quotes

```

// Request
{
  "customerId": "c1a2...",           // cliente registrado, nunca genérico
  "expirationDate": "2026-08-02",
  "discountAmount": 0,
  "notes": "Incluye entrega en obra.",
  "items": [
    { "productId": "p001...", "quantity": 1, "unitPrice": 1500.00 }
  ]
}

// Response 201
{ "id": "q555...", "quoteNumber": "COT-000078", "status": "PENDIENTE",

```

```
"subtotal": 1500.00, "taxAmount": 270.00, "total": 1770.00,
"stockWarnings": [] }
```

25.5 POST /api/v1/quotes/:id/convert

```
// Request
{ "initialPayment": { "paymentMethod": "CARD",
  "amount": 1770.00, "referenceNumber": "VISA-9912" } }

// Response 201
{ "saleId": "s777...", "saleNumber": "NV-000125",
  "quoteStatus": "CONVERTIDA", "paymentStatus": "PAID" }

// Response 409 (estado no convertible)
{ "statusCode": 409, "errorCode": "QUOTE_NOT_CONVERTIBLE",
  "message": "La cotización está VENCIDA y no puede convertirse." }
```

25.6 POST /api/v1/sales/:id/cancel

```
// Request
{ "reason": "Error en cantidades registradas por el vendedor." }

// Response 200
{ "id": "s9d8...", "saleStatus": "CANCELLED",
  "stockReverted": true, "reversalEntryNumber": "AST-000452" }
```

26. Manejo de autenticación JWT

- Emisión: al hacer login, el backend firma un JWT (HS256 con JWT_SECRET robusto) con payload mínimo: sub (user id), username, role y exp. Expiración recomendada para demo: 8 horas.
- Transporte: el token se entrega en cookie HttpOnly + Secure + SameSite=Lax (o Strict), nunca en localStorage. El frontend no puede leerlo por JavaScript, lo que mitiga XSS.
- Verificación: JwtAuthGuard global valida firma y expiración en cada request; la estrategia carga el usuario y verifica que su estado siga siendo ACTIVE (un usuario bloqueado pierde acceso aunque su token no haya expirado).
- Logout: endpoint que limpia la cookie (Set-Cookie con expiración pasada). Para MVP no se requiere lista de revocación; queda como mejora futura junto con refresh tokens.
- CSRF: al usar cookies, aplicar SameSite y, si se despliega en dominios distintos, token CSRF de doble envío. CORS restringido al dominio del frontend con credentials: true.
- Cambio de contraseña obligatorio: si must_change_password = true, el backend restringe el acceso al resto de endpoints hasta completar POST /auth/change-password.

27. Manejo de roles y permisos

La autorización se implementa con un decorador @Roles(...) y un RolesGuard que valida el rol del JWT contra el endpoint. La matriz siguiente reproduce las decisiones del Documento Maestro:

Módulo	ADMIN	SELLER	WAREHOUSE	MANAGEMENT
Dashboard	Total	Limitado	Limitado inventario	Total
Usuarios y roles	Total	No	No	Consulta opcional
Productos	Total	Consulta	Consulta/stock	Consulta
Categorías y unidades	Total	Consulta	Consulta	Consulta
Inventario	Total	No	Total	Consulta
Clientes	Total	Crear/editar	No	Consulta
Cotizaciones	Total	Total	No	Consulta

Módulo	ADMIN	SELLER	WAREHOUSE	MANAGEMENT
Ventas/POS	Total	Crear venta	No	Consulta
Pagos	Total	Registrar	No	Consulta
Reportes	Total	Limitado	Inventario	Total
Contabilidad básica	Total	No	No	Consulta
Auditoría	Total	No	No	Opcional
Configuración	Total	No	No	Consulta

- Los roles se precargan por seed; no hay matriz editable en MVP.
- Anulación de ventas y pagos: exclusiva de ADMIN.
- Las tablas permissions/role_permissions quedan opcionales para granularidad futura sin refactor.
- El frontend oculta opciones según rol solo como experiencia de usuario; la seguridad real es del backend.

28. Estrategia de seguridad

- Contraseñas con Argon2id (o bcrypt con costo ≥ 12); nunca texto plano ni cifrado reversible.
- JWT en cookie HttpOnly + Secure + SameSite; expiración de 8 horas en demo.
- Autorización por roles validada en backend en todos los endpoints.
- Validación de entrada global (ValidationPipe con whitelist y forbidNonWhitelisted) para rechazar campos no esperados.
- Rate limiting (@nestjs/throttler) en /auth/login y bloqueo de usuario tras superar el máximo de intentos fallidos configurado.
- Mensajes de error de login genéricos; sin enumeración de usuarios.
- CORS restringido al dominio del frontend; helmet para cabeceras seguras.
- HTTPS obligatorio en despliegue real; en demo local puede usarse HTTP tras aviso explícito.
- Prisma parametriza todas las consultas (previene inyección SQL); prohibido concatenar SQL crudo con entrada del usuario.
- Subida de imágenes: validar tipo MIME y tamaño máximo; renombrar archivos; nunca ejecutar contenido subido.
- Secretos únicamente por variables de entorno; .env fuera del repositorio (.env.example versionado).
- Auditoría de acciones críticas como control detectivo (ver sección 29).

29. Estrategia de auditoría

- Tabla única audit_logs con metadata JSONB; registro inmutable (solo INSERT y SELECT).
- AuditService centralizado invocado desde los casos de uso; para operaciones transaccionales, el log se inserta dentro de la misma transacción.
- Se auditan: LOGIN_SUCCESS, LOGIN_FAILED, USER_CREATED/UPDATED/BLOCKED, PASSWORD_RESET, PRODUCT_CREATED, PRODUCT_PRICE_CHANGED, PRODUCT_STATUS_CHANGED, INVENTORY_MOVEMENT, CUSTOMER_CREATED/UPDATED, QUOTE_CREATED, QUOTE_STATUS_CHANGED, QUOTE_CONVERTED, SALE_CREATED, SALE_CANCELLED, PAYMENT_REGISTERED, PAYMENT_CANCELLED, SETTINGS_UPDATED.
- No se auditan búsquedas, listados ni navegación (evita ruido y crecimiento innecesario).
- metadata guarda old/new en actualizaciones sensibles (precio, configuración, estado de usuario) y datos de contexto (totales, motivo, correlativo).
- Consulta solo para ADMIN (MANAGEMENT opcional) con filtros por fecha, usuario, módulo, acción y entidad.
- Índices sobre created_at, user_id y (module, action) para consulta eficiente.

30. Estrategia de manejo de errores

Formato único de error en toda la API, producido por un filtro global de excepciones:

```
{
  "statusCode": 422,
  "errorCode": "PAYMENT_EXCEEDS_BALANCE",
  "message": "El monto supera el saldo pendiente.",
  "details": [ ... ],           // opcional: errores de campo o contexto
  "timestamp": "2026-07-18T15:31:02Z",
  "path": "/api/v1/payments"
}
```

- Excepciones de negocio tipadas (AppException con errorCode) lanzadas desde casos de uso; el filtro las traduce a HTTP: 400 validación, 401 autenticación, 403 autorización, 404 no encontrado, 409 conflicto de estado, 422 regla de negocio.
- Errores de Prisma mapeados: P2002 (unicidad) → 409 con campo en conflicto; P2025 (no encontrado) → 404.
- Errores 500: mensaje genérico al cliente ("Ocurrió un error interno") y detalle completo solo en logs; nunca exponer stack trace ni SQL.
- El frontend centraliza el manejo en un interceptor HTTP: muestra message al usuario y usa errorCode para lógica específica (p. ej. redirigir si el token expiró).
- Los errores de validación de DTO retornan details con lista de campos y motivos.

31. Estrategia de logging

- Logger estructurado (Pino o Winston) con salida JSON en producción y legible en desarrollo.
- Niveles: error (fallos), warn (reglas de negocio rechazadas relevantes), info (arranque, operaciones críticas confirmadas), debug (solo desarrollo).
- Interceptor HTTP que registra método, ruta, usuario, código de estado y duración por request, con requestId correlacionable.
- Nunca loguear contraseñas, hashes, tokens, cookies ni cuerpos completos con datos sensibles.
- Los logs técnicos no reemplazan a audit_logs: el log es operativo/diagnóstico, la auditoría es funcional y persistente en base de datos.
- En Docker, logs a stdout/stderr para recolección estándar.

32. Estrategia de transacciones de base de datos

- Toda operación que afecta más de una tabla se ejecuta en prisma.\$transaction (API interactiva), incluyendo: crear venta, convertir cotización, registrar/anular pago, anular venta, registrar movimiento de inventario y generar correlativos.
- Los casos de uso reciben el cliente transaccional (tx) y lo propagan a los servicios transversales (InventoryService, AccountingService, SequenceService, AuditService) para que todo confirme o falle junto.
- Concurrencia de stock: SELECT ... FOR UPDATE sobre el producto (vía \$queryRaw parametrizado o updateMany condicional) antes de validar y descontar, evitando ventas simultáneas del mismo stock.
- Concurrencia de correlativos: UPDATE ... RETURNING sobre document_sequences dentro de la transacción; el bloqueo de fila garantiza números únicos y consecutivos.
- Nivel de aislamiento: READ COMMITTED de PostgreSQL es suficiente con los bloqueos anteriores; documentar cualquier excepción.
- Timeouts y reintentos: timeout explícito en transacciones; ante deadlock (40P01), reintento controlado del caso de uso.
- Nada de efectos externos dentro de la transacción (correo, archivos): primero commit, luego efectos secundarios.

33. Estrategia para ventas y reversión de stock

Confirmación de venta

- La cotización nunca descuenta ni reserva stock; solo la venta confirmada lo hace.
- Al confirmar: por cada ítem inventariable se bloquea el producto, se valida $\text{stock_current} \geq \text{quantity}$, se actualiza `stock_current` y se inserta un movimiento `SALIDA_VENTA` con `previous_stock`, `new_stock`, `reference_type = SALE` y `reference_id`.
- Los servicios (`is_inventory_tracked = false`) no generan movimientos.
- Si algún ítem no tiene stock suficiente, toda la venta se rechaza (rollback) con detalle de faltantes.

Anulación y reversa

- La venta confirmada no se edita: los errores se corrigen anulando y creando una nueva venta.
- Anular genera, en la misma transacción: movimientos `REVERSA_ANULACION` que devuelven el stock exacto de los `sale_items` inventariables, marcado de pagos según regla, asiento contable de reversión y auditoría con motivo.
- La reversa usa las cantidades del snapshot de `sale_items`, no el estado actual del producto, garantizando consistencia histórica.
- Una venta `ANULADA` no puede recibir pagos ni anularse de nuevo (idempotencia por validación de estado).

34. Estrategia para pagos completos y parciales

- Modelo: una venta tiene N pagos; `paid_amount` = suma de pagos `ACTIVE`; `balance_due` = `total` – `paid_amount`; `payment_status` derivado: `UNPAID` (0), `PARTIALLY_PAID` ($0 < \text{pagado} < \text{total}$), `PAID` (`pagado = total`).
- Pago completo al crear la venta es el flujo principal del POS; el pago parcial genera saldo y la venta aparece automáticamente en cuentas por cobrar.
- Pagos posteriores se registran sobre la venta con validación de saldo dentro de transacción (bloqueo de la venta para evitar sobrepago concurrente).
- Los pagos no se editan: se anulan con motivo (`CANCELLED`) y, si corresponde, se registra uno nuevo. La anulación recalcula saldo y estado, y genera reversión contable.
- Métodos soportados: `CASH`, `BANK_TRANSFER`, `BANK_DEPOSIT`, `CARD`, `DIGITAL_WALLET`, `OTHER`. Transferencia, depósito y tarjeta exigen `reference_number`.
- Regla de crédito: una venta que quedará con saldo exige cliente registrado y no `BLOCKED`; el cliente genérico solo participa en ventas pagadas al contado.

35. Estrategia para contabilidad básica / precontabilidad

El módulo es precontabilidad interna automática: da trazabilidad económica a ventas y cobros sin pretender ser contabilidad tributaria. No genera libros oficiales, no tiene asientos manuales, ni PLE, ni conciliación bancaria, ni integración externa.

Plan de cuentas básico (seed)

Código	Cuenta	Tipo
101	Caja	ASSET
104	Bancos	ASSET
121	Cuentas por cobrar	ASSET
401	IGV por pagar	LIABILITY

Código	Cuenta	Tipo
701	Ventas	INCOME
741	Descuentos concedidos	INCOME (contra-cuenta)

Asientos automáticos

Evento	Debe	Haber
Venta confirmada (SALE)	Cuentas por cobrar: total	Ventas: base imponible · IGV por pagar: impuesto (si activo)
Pago registrado (PAYMENT)	Caja (CASH) o Bancos (demás métodos): monto	Cuentas por cobrar: monto
Anulación de venta	Asiento inverso del asiento SALE original	—
Anulación de pago	Asiento inverso del asiento PAYMENT original	—

- El descuento global se registra rebajando la base de Ventas (o con línea en Descuentos concedidos, decisión a fijar en la fase 8; por defecto: base neta de descuento).
- Toda reversión se vincula con `reversed_entry_id` y marca `is_reversed` en el asiento original; los asientos jamás se eliminan.
- Validación dura: `total_debit = total_credit`; si no cuadra, la operación completa (venta/pago) se revierte.
- Consulta: libro diario interno con filtros y detalle de líneas, solo ADMIN y MANAGEMENT.

36. Estrategia de correlativos

Documento	document_type	Prefijo	Formato ejemplo
Cotización	QUOTE	COT	COT-000078
Nota de venta interna	SALE	NV	NV-000124
Pago	PAYMENT	PAG	PAG-000310
Asiento contable	ACCOUNTING_ENTRY	AST	AST-000452

- `SequenceService.next(documentType, tx)`: dentro de la transacción del documento ejecuta `UPDATE document_sequences SET current_number = current_number + 1 WHERE document_type = $1 RETURNING prefix, current_number, padding` y formatea el número.
- El bloqueo de fila de PostgreSQL garantiza unicidad bajo concurrencia; además, la columna del documento (`quote_number`, `sale_number`, etc.) tiene índice UNIQUE como segunda barrera.
- Si la transacción del documento falla, el consumo del número se revierte con el rollback (sin huecos en demo).
- El frontend nunca genera ni sugiere números de documento.
- La nota de venta interna con correlativo NV no es comprobante fiscal y debe indicarlo en su impresión.

37. Estrategia de reportes y dashboard

Dashboard por rol

- `GET /dashboard/summary` arma tarjetas, gráficos y listas cortas según el rol: ADMIN/MANAGEMENT ven ventas del día/mes, cobros, cuentas por cobrar, stock bajo y cotizaciones por estado; SELLER ve sus ventas y cotizaciones; WAREHOUSE ve stock bajo y últimos movimientos.
- Consultas agregadas directas con SQL/Prisma sobre índices por fecha y estado; sin tablas de precálculo en MVP (volumen de demo lo permite).

Los diez reportes básicos

N°	Reporte	Endpoint sugerido	Filtros principales	Columnas clave
1	Ventas por fecha	/reports/sales-by-date	Fecha, estado, vendedor, cliente	Fecha, N° venta, cliente, vendedor, total, pagado, saldo, estado
2	Ventas por producto	/reports/sales-by-product	Fecha, categoría, producto	SKU, producto, categoría, cantidad vendida, total vendido
3	Ventas por cliente	/reports/sales-by-customer	Fecha, cliente, tipo cliente	Cliente, documento, cantidad ventas, total, pagado, saldo
4	Ventas por vendedor	/reports/sales-by-seller	Fecha, vendedor	Vendedor, cantidad ventas, total vendido, cobrado, cotizaciones convertidas
5	Productos con stock bajo	/reports/low-stock	Categoría, marca, estado	SKU, producto, stock actual, mínimo, diferencia
6	Inventario actual	/reports/inventory	Categoría, marca, stock bajo	SKU, producto, unidad, stock, mínimo, estado
7	Movimientos de inventario	/reports/inventory-movements	Fecha, producto, tipo, usuario	Fecha, SKU, tipo, cantidad, stock anterior/nuevo, motivo
8	Cotizaciones por estado	/reports/quotes-by-status	Fecha, estado, vendedor, cliente	N° cotización, cliente, total, estado, venta generada
9	Pagos por método	/reports/payments-by-method	Fecha, método, estado, usuario	Fecha, pago, venta, cliente, método, referencia, monto
10	Cuentas por cobrar	/reports/receivables	Cliente, fecha, vendedor	Cliente, venta, total, pagado, saldo, días pendiente

- Regla transversal: ventas y pagos anulados quedan excluidos de los totales principales; los productos no inventariables no aparecen en stock bajo.
- Autorización por rol aplicada en backend a cada reporte según la matriz de la sección 27.
- Exportación Excel/PDF es opcional en MVP; la prioridad es la visualización en pantalla con tabla, filtros y totales.

38. Variables de entorno necesarias

Las variables se documentan en .env.example (versionado) y se validan al arranque del backend (fallar rápido si falta alguna obligatoria). Los valores reales nunca se versionan.

Variable	Ejemplo (demo)	Descripción
NODE_ENV	development	Entorno: development production.
PORT	3000	Puerto del backend NestJS.
DATABASE_URL	postgresql://pos:***@db:5432/pos_db?schema=public	Cadena de conexión Prisma/PostgreSQL.
JWT_SECRET	(cadena aleatoria ≥ 32 caracteres)	Firma de tokens JWT. Obligatoria y robusta.
JWT_EXPIRES_IN	8h	Expiración del token (8 horas en demo).
COOKIE_SECURE	false (demo) / true (producción)	Flag Secure de la cookie de sesión.
CORS_ORIGIN	http://localhost:4200	Dominio permitido del frontend.
LOGIN_MAX_ATTEMPTS	5	Intentos fallidos antes de BLOCKED.
THROTTLE_TTL / THROTTLE_LIMIT	60 / 10	Rate limiting de login (ventana en segundos / peticiones).

Variable	Ejemplo (demo)	Descripción
FILE_STORAGE_DRIVER	local	local (demo) s3 (producción).
FILE_STORAGE_PATH	./storage/uploads	Ruta local de archivos en demo.
S3_ENDPOINT / S3_BUCKET / S3_ACCESS_KEY / S3_SECRET_KEY	(solo producción)	Credenciales de almacenamiento S3 compatible.
MAX_UPLOAD_SIZE_MB	5	Tamaño máximo de imagen.
LOG_LEVEL	debug (dev) / info (prod)	Nivel de logging.
SEED_ADMIN_PASSWORD	(solo entorno demo)	Contraseña inicial del usuario admin del seed.
POSTGRES_USER / POSTGRES_PASSWORD / POSTGRES_DB	pos / *** / pos_db	Variables del contenedor PostgreSQL en docker-compose.

Frontend Angular: environment.ts define apiBaseUrl (p. ej. `http://localhost:3000/api/v1`); no contiene secretos.

39. Plan de migraciones inicial

Las migraciones se gestionan con Prisma Migrate (`prisma migrate dev / deploy`), versionadas en el repositorio. Orden inicial recomendado, respetando dependencias de claves foráneas:

Nº	Migración	Tablas / objetos	Depende de
1	init_security	roles, users (+ permissions, role_permissions opcionales)	—
2	init_catalog	categories, units, products, product_specifications, product_images	1
3	init_inventory	inventory_movements + índices por product_id y created_at	1, 2
4	init_customers	customers + índice único (document_type, document_number)	—
5	init_quotes	quotes, quote_items + índice único quote_number	1, 2, 4
6	init_sales	sales, sale_items + FK quotes.converted_sale_id + índices por fecha/estado	1, 2, 4, 5
7	init_payments	payments + índice único payment_number	1, 6
8	init_accounting	chart_of_accounts, accounting_entries, accounting_entry_lines	—
9	init_settings	company_settings, document_sequences (filas QUOTE, SALE, PAYMENT, ACCOUNTING_ENTRY)	—
10	init_audit	audit_logs + índices de consulta	1
11	constraints_hardening	CHECKs de montos/stock/debe=haber que Prisma no cubre (SQL en migración manual)	1–10

- Cada migración debe ser aplicable sobre base vacía y reproducible en CI/Docker.
- Los CHECK avanzados (`total_debit = total_credit`, `new_stock >= 0`) se agregan como SQL dentro de una migración de Prisma (`migration.sql` editada).
- Regla del proyecto: nunca editar migraciones ya aplicadas en otros entornos; siempre crear una nueva.

40. Datos semilla para demo

`prisma/seed.ts` crea los datos mínimos para una demo presentable, sin datos reales ni branding de una empresa específica. El seed debe ser idempotente (`upsert` por códigos únicos).

Tipo	Registros semilla
Roles	ADMIN, SELLER, WAREHOUSE, MANAGEMENT.
Usuarios	admin (ADMIN), vendedor01 (SELLER), almacen01 (WAREHOUSE), gerencia01 (MANAGEMENT), bloqueado01 (SELLER, estado BLOCKED para demostrar la regla). Todos con must_change_password según convenga a la demo.
Configuración	Empresa demo genérica, moneda PEN (S/), IGV activo 18%, vigencia de cotización 15 días, correlativos COT/NV/PAG/AST en 0.
Categorías	Máquinas de construcción, Gasfitería y tuberías, Sistemas de agua, Equipos hidráulicos, Pintura, Repuestos, Servicios (con al menos una subcategoría de ejemplo).
Unidades	UND, MTR, KG, LTR, GLN, CJ, JGO, PAR, RLL, SER (SER con allow_decimal = false).
Productos	Generador 3.300 KW, Cámara de inspección 30 m, Bomba de agua 2", Manguera hidráulica (unidad MTR, allow_decimal = true), Boquilla de repuesto y Servicio técnico (SERVICE, no inventariable). Con especificaciones, stock inicial vía movimientos INGRESO_MANUAL y algún producto en stock bajo.
Clientes	Público general (is_generic = true), Constructora San Miguel S.A.C., Servicios Industriales del Norte S.A.C., Carlos Mendoza (PERSON), Grupo Técnico Andino S.A.C. (uno como PROSPECT).
Plan de cuentas	Caja, Bancos, Cuentas por cobrar, IGV por pagar, Ventas, Descuentos concedidos.
Cotizaciones	Una por estado: PENDIENTE, ACEPTADA, RECHAZADA, VENCIDA y CONVERTIDA (vinculada a su venta).
Ventas	Casos: PAGADA (contado), PAGADA_PARCIAL (con saldo), PENDIENTE_PAGO y ANULADA (con reversa de stock y asiento de reversión).
Pagos	Efectivo, transferencia (con referencia), tarjeta, billetera digital y un pago anulado.
Auditoría	Generada naturalmente por las operaciones del propio seed (login no; operaciones sí).

Importante: el stock inicial se crea mediante movimientos INGRESO_MANUAL (no asignando stock_current directamente), para que la demo respete la regla "el stock solo cambia por movimientos".

41. Plan de desarrollo por fases

El orden respeta las dependencias funcionales del Documento Maestro. Cada fase termina con su módulo funcionando de punta a punta (API + pantalla + validaciones + auditoría cuando aplique).

Fase	Módulos / actividad	Resultado esperado
0	Definición técnica inicial: repositorios, Docker Compose, base de datos, Prisma, migraciones, esqueleto NestJS/Angular, CI básico.	Proyecto listo para desarrollo; entorno reproducible.
1	Autenticación, usuarios, roles y layout base con navegación por rol.	Acceso seguro; guards funcionando; reset de contraseña.
2	Categorías, unidades y productos (con especificaciones e imágenes).	Catálogo funcional con validaciones.
3	Inventario: movimientos, ajustes, historial y stock bajo.	Stock trazable por movimientos; sin stock negativo.
4	Clientes: registro, búsqueda, registro rápido y detalle.	Clientes y prospectos operativos.
5	Cotizaciones: ítems con snapshot, estados, PDF simple y base de conversión.	Cotizaciones completas y presentables.
6	Ventas / POS: venta directa, desde cotización, validación de stock, nota interna y anulación.	Flujo de venta transaccional completo.
7	Pagos: completos/parciales, anulación, saldo y cuentas por cobrar.	Cobranza operativa con saldos correctos.
8	Contabilidad básica: asientos automáticos, reversiones y libro diario interno.	Precontabilidad cuadrada y trazable.

Fase	Módulos / actividad	Resultado esperado
9	Dashboard y los diez reportes básicos.	Indicadores y reportes filtrables por rol.
10	Configuración y auditoría: parámetros, correlativos visibles y consulta de auditoría.	Parametrización y trazabilidad completas.
11	Ajustes de mockups finales, pruebas de demo y datos semilla definitivos.	Demo preparada para presentación.

41.1 Tabla resumen del plan (Fase · Módulo · Actividad · Prioridad · Dependencia · Resultado · Estado)

Fase	Módulo	Actividad técnica	Prioridad	Dependencia	Resultado esperado	Estado inicial sugerido
0	Infraestructura	Crear repos, docker-compose (frontend, backend, PostgreSQL), esqueleto NestJS y Angular, Prisma configurado.	Alta	—	Entorno reproducible levantando con un comando.	Por iniciar
0	Base de datos	Ejecutar migraciones 1–11 y validar schema.prisma contra la sección 18.	Alta	Infraestructura	Esquema completo con restricciones.	Por iniciar
0	Transversales	GlobalExceptionHandler, ValidationPipe, logger, config de entorno validada, Swagger.	Alta	Infraestructura	Base común de errores, validación y logs.	Por iniciar
1	Autenticación	Login/logout JWT en cookie HttpOnly, guards, bloqueo por intentos, must_change_password.	Alta	Fase 0	Acceso seguro validado en backend.	Por iniciar
1	Usuarios y roles	CRUD de usuarios, reset por ADMIN, seed de roles, protección del único ADMIN activo.	Alta	Autenticación	Gestión de usuarios operativa.	Por iniciar
1	Frontend base	Layout, sidebar por rol, interceptores HTTP y pantalla de login.	Alta	Autenticación	Navegación protegida por rol.	Por iniciar
2	Categorías y unidades	CRUD con jerarquía sin ciclos, allow_decimal y baja lógica.	Alta	Fase 1	Estructura del catálogo lista.	Por iniciar
2	Productos	CRUD con SKU único, tipo, is_inventory_tracked, especificaciones e imágenes.	Alta	Categorías y unidades	Catálogo técnico funcional.	Por iniciar
3	Inventario	Casos de uso de movimientos con FOR UPDATE, ajustes con motivo, historial y stock bajo.	Alta	Productos	Stock consistente y trazable.	Por iniciar
4	Clientes	CRUD con unicidad por documento, registro rápido, cliente genérico y estados.	Alta	Fase 1	Base de clientes lista para ventas.	Por iniciar
5	Cotizaciones	Creación con snapshot y correlativo COT, estados, vencimiento y PDF simple.	Alta	Productos, Clientes	Cotizaciones completas.	Por iniciar
6	Ventas / POS	Caso de uso create-sale transaccional (stock + correlativo NV + pago inicial + asiento), pantalla POS y anulación con reversas.	Alta	Inventario, Clientes, Cotizaciones	Venta de punta a punta con nota interna.	Por iniciar

Fase	Módulo	Actividad técnica	Prioridad	Dependencia	Resultado esperado	Estado inicial sugerido
6	Conversión	convert-quote-to-sale reutilizando create-sale y marcando CONVERTIDA.	Alta	Ventas	Conversión transaccional correcta.	Por iniciar
7	Pagos	register-payment y cancel-payment con recalcule de saldo y cuentas por cobrar.	Alta	Ventas	Cobranza completa.	Por iniciar
8	Contabilidad básica	AccountingService: asientos SALE/PAYMENT/reversiones, plan de cuentas seed y libro diario.	Media/Alta	Ventas, Pagos	Asientos automáticos cuadrados.	Por iniciar
9	Dashboard	Summary por rol con tarjetas, gráficos y listas cortas.	Alta	Fases 6–8	Indicadores tras login.	Por iniciar
9	Reportes	Diez endpoints de reportes con filtros y exclusión de anulados.	Alta	Fases 6–8	Reportes filtrables por rol.	Por iniciar
10	Configuración	Pantalla de empresa, moneda, IGV, parámetros y correlativos (auditada).	Alta	Fase 1	Parametrización operativa.	Por iniciar
10	Auditoría	Consulta con filtros y detalle de metadata (solo lectura).	Media	Fases 1–8	Trazabilidad consultable.	Por iniciar
11	Demo	Seed definitivo, pruebas del checklist de demo, pulido de mockups y guion de presentación.	Alta	Todas	Demo lista para presentar.	Por iniciar

42. Backlog técnico priorizado

Backlog ordenado por prioridad de ejecución (P1 = bloqueante para lo demás). Cada ítem corresponde a una o más historias de la sección 43.

#	Ítem de backlog	Prioridad	Fase
1	Docker Compose + esqueletos NestJS/Angular + Prisma + migraciones iniciales	P1	0
2	Infra transversal: errores, validación global, logger, Swagger, config validada	P1	0
3	Autenticación JWT en cookie + guards + bloqueo por intentos	P1	1
4	CRUD usuarios y roles + reset de contraseña + seed de roles	P1	1
5	Layout Angular con navegación por rol e interceptores	P1	1
6	CRUD categorías (jerarquía) y unidades (allow_decimal)	P1	2
7	CRUD productos + especificaciones + imágenes + validación SKU	P1	2
8	Casos de uso de inventario (movimientos, ajustes) + pantallas + stock bajo	P1	3
9	CRUD clientes + registro rápido + cliente genérico + unicidad de documento	P1	4
10	Cotizaciones: creación, estados, correlativo COT, PDF simple	P1	5
11	SequenceService transaccional (COT/NV/PAG/AST)	P1	5
12	create-sale transaccional (stock + pago inicial + asiento) + pantalla POS	P1	6
13	convert-quote-to-sale + marcado CONVERTIDA	P1	6

#	Ítem de backlog	Prioridad	Fase
14	cancel-sale con reversa de stock, pagos y contabilidad	P1	6
15	register-payment / cancel-payment + cuentas por cobrar	P1	7
16	AccountingService + plan de cuentas + libro diario interno	P2	8
17	Dashboard summary por rol	P2	9
18	Diez reportes básicos con filtros	P2	9
19	Configuración de empresa/moneda/IGV/parámetros (auditada)	P2	10
20	Consulta de auditoría con filtros	P2	10
21	Seed completo de demo + pruebas del checklist + pulido visual	P2	11
22	Opcionales post-demo: exportación Excel/PDF de reportes, job de vencimiento de cotizaciones	P3	—

43. Historias técnicas por sprint

Planificación de referencia en 6 sprints de 2 semanas (ajustable al equipo). Formato: "Como <rol técnico>, quiero <capacidad>, para <resultado>".

Sprint 1 — Fundaciones y acceso (Fases 0–1)

- Como equipo, quiero levantar frontend, backend y PostgreSQL con docker-compose up, para tener un entorno reproducible.
- Como equipo, quiero migraciones Prisma del esquema completo con restricciones, para asegurar integridad desde el día uno.
- Como desarrollador backend, quiero filtro global de errores, ValidationPipe con whitelist, logger y Swagger, para estandarizar la API.
- Como usuario interno, quiero iniciar sesión con JWT en cookie HttpOnly y ser bloqueado tras N intentos fallidos, para acceder de forma segura.
- Como ADMIN, quiero crear/editar/bloquear usuarios y resetear contraseñas con must_change_password, para administrar el acceso.
- Como usuario, quiero un layout con menú según mi rol, para navegar solo lo permitido.

Sprint 2 — Catálogo (Fase 2)

- Como ADMIN, quiero CRUD de categorías jerárquicas sin ciclos y de unidades con allow_decimal, para estructurar el catálogo.
- Como ADMIN, quiero crear productos con SKU único, tipo PRODUCT/SERVICE, is_inventory_tracked, descripciones separadas, especificaciones e imágenes, para administrar el catálogo técnico.
- Como usuario, quiero buscar productos por texto, categoría, marca y estado con paginación, para encontrar ítems rápido.
- Como sistema, quiero auditar creación y cambios de precio/estado de productos, para trazabilidad.

Sprint 3 — Inventario y clientes (Fases 3–4)

- Como WAREHOUSE, quiero registrar ingresos manuales y ajustes con motivo obligatorio, para mantener el stock real.
- Como sistema, quiero bloquear el producto y validar stock en transacción guardando previous_stock/new_stock, para impedir stock negativo y carreras.
- Como WAREHOUSE, quiero ver historial de movimientos filtrable y stock bajo, para controlar el inventario.
- Como SELLER, quiero registrar clientes (completo y rápido) con unicidad por documento, para vender y cotizar.

- Como sistema, quiero impedir que clientes BLOCKED o el genérico queden con saldo pendiente, para controlar el crédito.

Sprint 4 — Cotizaciones y ventas (Fases 5–6)

- Como SELLER, quiero crear cotizaciones con snapshot de producto, correlativo COT, vigencia y PDF simple, para enviar propuestas.
- Como SELLER, quiero marcar cotizaciones ACEPTADA/RECHAZADA y que venzan automáticamente, para gestionar el pipeline.
- Como SELLER, quiero registrar venta directa en POS con carrito, validación de stock y pago inicial, para vender al contado o al crédito.
- Como sistema, quiero ejecutar la venta en una sola transacción (correlativo NV, snapshot, SALIDA_VENTA, pago, asiento, auditoría), para consistencia total.
- Como SELLER, quiero convertir cotizaciones vigentes en ventas, para cerrar propuestas sin redigitar.
- Como ADMIN, quiero anular ventas con motivo, revirtiendo stock, pagos y contabilidad, para corregir errores de forma controlada.

Sprint 5 — Pagos y contabilidad (Fases 7–8)

- Como SELLER, quiero registrar pagos completos y parciales con métodos y referencia obligatoria cuando aplique, para cobrar ventas.
- Como sistema, quiero recalcular paid_amount, balance_due y payment_status en transacción, para saldos siempre correctos.
- Como ADMIN, quiero anular pagos con motivo y reversión contable, para corregir cobros.
- Como MANAGEMENT, quiero ver cuentas por cobrar con días pendientes, para gestionar la cobranza.
- Como sistema, quiero generar asientos SALE/PAYMENT/reversión cuadrados y vinculados por reversed_entry_id, para precontabilidad trazable.
- Como MANAGEMENT, quiero consultar el libro diario interno con detalle de líneas, para revisar la precontabilidad.

Sprint 6 — Reportes, configuración y demo (Fases 9–11)

- Como usuario, quiero un dashboard por rol tras el login, para ver indicadores clave.
- Como MANAGEMENT, quiero los diez reportes básicos con filtros y totales que excluyan anulados, para tomar decisiones.
- Como ADMIN, quiero editar configuración de empresa, moneda, IGV y parámetros con auditoría old/new, para parametrizar el sistema.
- Como ADMIN, quiero consultar la auditoría con filtros y ver la metadata, para investigar acciones críticas.
- Como equipo, quiero el seed de demo completo y el checklist de demo en verde, para presentar el MVP.

44. Criterios de aceptación técnicos

- Un usuario BLOCKED o INACTIVE no puede iniciar sesión; el mensaje de error no revela si el usuario existe.
- Tras N intentos fallidos configurados, el usuario pasa a BLOCKED y queda auditado.
- Todo endpoint (salvo login) rechaza con 401 sin JWT válido y con 403 si el rol no está autorizado, aunque el frontend oculte la opción.
- No es posible crear dos productos con el mismo SKU ni dos clientes con el mismo (tipo, número de documento): la API responde 409.
- El stock nunca queda negativo: ventas y ajustes negativos con stock insuficiente responden 422 con detalle, incluso bajo peticiones concurrentes.
- Cada movimiento de inventario persiste previous_stock y new_stock coherentes con el stock del producto.

- Una cotización a cliente genérico es rechazada; una cotización CONVERTIDA/RECHAZADA/VENCIDA no puede convertirse.
- Confirmar una venta genera, en una sola transacción: correlativo NV único, snapshot de ítems, movimientos SALIDA_VENTA, pago inicial (si se envió), asiento SALE cuadrado y registro de auditoría; si algo falla, no queda ningún efecto parcial.
- Una venta con saldo pendiente no puede registrarse con el cliente Público general (422).
- Un pago mayor al saldo, con monto ≤ 0 , sin referencia cuando el método la exige, o sobre venta anulada, es rechazado.
- Anular una venta devuelve el stock exacto, genera asiento de reversión vinculado (reversed_entry_id) y bloquea nuevos pagos.
- Todos los asientos cumplen total_debit = total_credit; el libro diario los muestra con su operación origen.
- Los correlativos COT/NV/PAG/AST no se repiten bajo concurrencia (verificable con prueba de carga simple).
- Los reportes excluyen ventas y pagos anulados de los totales, y respetan la matriz de permisos por rol.
- La auditoría no expone endpoints de edición ni eliminación.
- docker-compose up + migraciones + seed dejan el sistema demo funcionando de punta a punta.

45. Checklist para iniciar desarrollo

- Repositorios creados (backend y frontend o monorepo) con rama principal protegida y convención de commits.
- docker-compose.yml levanta PostgreSQL, backend y frontend; README con instrucciones de un solo comando.
- .env.example completo; validación de entorno al arranque del backend.
- schema.prisma refleja la sección 18; migraciones 1–11 aplicadas sobre base vacía.
- Seed mínimo de roles + usuario admin funcionando.
- ValidationPipe global, filtro global de excepciones con formato único y logger configurados.
- Swagger disponible en /docs (solo entornos no productivos).
- JwtAuthGuard y RolesGuard registrados; decoradores @Roles y @CurrentUser listos.
- SequenceService, AuditService, InventoryService y AccountingService creados como esqueleto con sus contratos.
- Angular con layout base, rutas lazy por feature, guards e interceptores (auth + errores).
- ESLint + Prettier + Jest configurados en ambos proyectos; pipeline CI ejecuta lint, build y pruebas.
- Definición de terminado (DoD) acordada: código revisado, validaciones en backend, auditoría donde aplique y criterio de aceptación verificado.

46. Checklist para probar la demo

- Login correcto con admin, vendedor01, almacen01 y gerencia01; login rechazado con bloqueado01 y con contraseña errónea (mensaje genérico).
- Cada rol ve solo su menú y sus datos según la matriz de permisos (verificar también por API directa).
- Crear un producto inventariable nuevo y verificar SKU duplicado rechazado.
- Registrar INGRESO_MANUAL y un AJUSTE_NEGATIVO con motivo; verificar previous_stock/new_stock y que un ajuste mayor al stock sea rechazado.
- Ver stock bajo con el producto sembrado bajo mínimo; confirmar que los servicios no aparecen.
- Crear cliente nuevo y verificar duplicado por documento rechazado.
- Crear cotización a cliente registrado, generar su PDF y verificar que a Público general no se puede cotizar.

- Convertir la cotización ACEPTADA a venta con pago con tarjeta (referencia obligatoria); verificar estado CONVERTIDA y stock descontado.
- Venta directa al contado con Público general en POS: correlativo NV, nota interna imprimible y asientos SALE + PAYMENT cuadrados.
- Venta al crédito: verificar que exige cliente registrado; registrar pago parcial y comprobar saldo, estado PARTIALLY_PAID y aparición en cuentas por cobrar.
- Intentar pagar más que el saldo (rechazado) y completar el pago (estado PAID).
- Anular un pago con motivo: saldo recalculado y asiento de reversión generado.
- Anular una venta con motivo: stock devuelto (movimientos REVERSA_ANULACION), venta sin opción de pago y asiento de reversión vinculado.
- Revisar libro diario: todos los asientos cuadran y enlazan a su operación origen.
- Recorrer los diez reportes con filtros; verificar que anulados no suman en totales.
- Dashboard por rol muestra indicadores coherentes con los datos de la demo.
- Cambiar un parámetro de configuración (p. ej. vigencia de cotización) y verificar auditoría con old/new.
- Consultar auditoría filtrando por módulo VENTAS y acción SALE_CANCELLED.

47. Riesgos técnicos y funcionales

Riesgo	Tipo	Impacto	Probabilidad	Mitigación
Condiciones de carrera en stock o correlativos (ventas simultáneas).	Técnico	Alto	Media	Bloqueo de fila (FOR UPDATE / UPDATE RETURNING) dentro de transacciones + índices UNIQUE como segunda barrera + prueba de concurrencia.
Inconsistencia entre venta, stock, pagos y asientos por transacciones mal delimitadas.	Técnico	Alto	Media	Casos de uso únicos por operación que reciben el cliente transaccional; prohibido llamar servicios fuera de la transacción; pruebas de rollback.
Scope creep: presión por agregar alquiler, facturación SUNAT, compras o kardex al MVP.	Funcional	Alto	Alta	Este documento y el Documento Maestro como fuente de verdad; todo cambio pasa por control de versiones del documento.
Asientos descuadrados por redondeos de IGV/descuento.	Técnico	Medio	Media	Redondeo definido a 2 decimales en un único servicio de cálculo; CHECK debe = haber; pruebas unitarias de cálculo.
Autorización solo en frontend (botones ocultos) dejando endpoints expuestos.	Técnico	Alto	Media	RolesGuard obligatorio por endpoint + pruebas de acceso por rol vía API en el checklist.
Snapshot incompleto: cambios de precio/nombre alteran documentos históricos.	Funcional	Medio	Media	quote_items y sale_items guardan SKU, nombre, unidad y precio; los reportes históricos leen el snapshot.
Demora en PDF (cotización/nota) por elegir herramienta tarde.	Técnico	Medio	Media	Decidir pdfmake o HTML→PDF en fase 5 con plantilla simple; el diseño fino queda para el final.
Seed no reproducible que rompe la demo.	Técnico	Medio	Baja	Seed idempotente (upsert), ejecutado en CI y verificado con el checklist de demo.
Confusión del usuario final creyendo que la nota de venta es comprobante fiscal.	Funcional	Medio	Media	Leyenda "Documento interno – no es comprobante fiscal" en impresión y capacitación en demo.
Crecimiento de audit_logs degradando consultas.	Técnico	Bajo	Baja	Auditar solo acciones críticas, índices adecuados y política de archivado post-MVP.

Riesgo	Tipo	Impacto	Probabilidad	Mitigación
Dependencia de una sola persona en módulos críticos.	Gestión	Medio	Media	Documentación por caso de uso, revisiones de código cruzadas y este documento como referencia.

48. Recomendaciones para mockups y pantallas

Antes de programar fuerte, generar wireframes funcionales de baja fidelidad (Figma, Excalidraw o similar). No se necesita diseño final perfecto: sí una estructura clara por pantalla, consistente con el layout por rol.

Pantalla	Debe incluir
Login	Usuario/correo, contraseña, error genérico, botón de ingreso y flujo de cambio de contraseña obligatorio.
Dashboard	Tarjetas, gráficos y tablas cortas según rol (ventas, cobros, stock bajo, cotizaciones, cuentas por cobrar).
Productos	Listado con filtros, nuevo producto, editar, detalle con especificaciones e imágenes, badge de estado y stock.
Categorías / Unidades	CRUD simple, estado, jerarquía visual de categorías y allow_decimal en unidades.
Inventario	Stock actual, nuevo movimiento (tipo, cantidad, motivo), historial filtrable y vista de stock bajo.
Clientes	Listado con búsqueda, crear/editar, registro rápido (modal), detalle con ventas, cotizaciones y saldo.
Cotizaciones	Listado por estado, crear con buscador de productos, detalle, cambiar estado, convertir a venta y PDF simple.
Ventas / POS	Selector de cliente (con registro rápido y Público general), buscador de productos, carrito, totales con descuento e IGV, panel de pago y confirmación.
Detalle de venta	Ítems (snapshot), pagos, movimientos de stock, estados (pago/entrega), botón anular (solo ADMIN) e impresión de nota interna.
Pagos	Lista de pagos, registrar pago sobre venta con saldo, anular pago y vista de cuentas por cobrar.
Contabilidad básica	Libro diario con filtros, detalle de asiento con líneas debe/haber y enlace a la operación origen.
Reportes	Selector de reporte, panel de filtros, tabla con paginación y fila de totales.
Configuración	Empresa (datos y logo), moneda, IGV, parámetros comerciales y estado de correlativos.
Auditoría	Filtros (fecha, usuario, módulo, acción), listado y detalle de metadata en formato legible.
Usuarios	Listado, crear, editar, reset de contraseña, bloquear/desbloquear y estado visible.

- Reutilizar componentes compartidos: buscador de producto, buscador/registro rápido de cliente, tabla paginada, badge de estado y diálogo de confirmación con motivo.
- Diseñar primero POS, detalle de venta y cotización: son las pantallas que definen la demo.
- Mantener lenguaje visual sobrio de sistema interno (sin estética de ecommerce ni de minimarket).

49. Recomendación final de cómo empezar a programar

Ruta concreta para la primera semana:

- Día 1: crear repositorios y docker-compose con PostgreSQL; generar esqueletos NestJS y Angular; configurar Prisma y ESLint/Prettier.
- Día 2: escribir schema.prisma completo a partir de la sección 18 y ejecutar las migraciones; agregar los CHECK por SQL; seed de roles + admin.

- Día 3: infraestructura transversal (filtro de errores, ValidationPipe, logger, Swagger, config validada) y esqueletos de SequenceService/AuditService.
- Días 4–5: módulo de autenticación completo (login/logout/me/change-password, guards, bloqueo por intentos) con pantalla de login y layout por rol.
- Semana 2 en adelante: seguir estrictamente las fases 2 → 11; no iniciar una fase sin cerrar los criterios de aceptación de la anterior.
- Principio rector: primero el flujo de negocio de punta a punta (cotización → venta → pago → asiento) con interfaz sobria, y solo después pulir estética.
- Escribir pruebas unitarias mínimas de los casos de uso críticos (create-sale, register-payment, cancel-sale, correlativos concurrentes) desde que se implementan.
- Cualquier duda funcional se resuelve contra el Documento Maestro; si la respuesta no está ahí, se decide, se documenta en el control de versiones y recién se programa.

50. Anexos técnicos

Anexo A. Catálogo de enumeraciones

Dominio	Valores
users.status	ACTIVE, INACTIVE, BLOCKED
roles.code	ADMIN, SELLER, WAREHOUSE, MANAGEMENT
products.product_type	PRODUCT, SERVICE
inventory_movements.movement_type	INGRESO_MANUAL, AJUSTE_POSITIVO, AJUSTE_NEGATIVO, SALIDA_VENTA, REVERSA_ANULACION
customers.customer_type / customer_stage	PERSON, COMPANY / PROSPECT, CUSTOMER
quotes.status	PENDIENTE, ACEPTADA, RECHAZADA, VENCIDA, CONVERTIDA
sales.payment_status	UNPAID, PARTIALLY_PAID, PAID
sales.sale_status	CONFIRMED, CANCELLED
sales.delivery_status	NO_APLICA, PENDIENTE, ENTREGADA, OBSERVADA
payments.payment_method	CASH, BANK_TRANSFER, BANK_DEPOSIT, CARD, DIGITAL_WALLET, OTHER
payments.status	ACTIVE, CANCELLED
accounting_entries.source_type	SALE, PAYMENT, SALE_CANCELLATION, PAYMENT_CANCELLATION
document_sequences.document_type	QUOTE, SALE, PAYMENT, ACCOUNTING_ENTRY

Anexo B. Códigos de error de negocio sugeridos

errorCode	HTTP	Situación
AUTH_INVALID_CREDENTIALS	401	Credenciales incorrectas o usuario no ACTIVE (mensaje genérico).
AUTH_USER_BLOCKED	401	Usuario bloqueado por intentos (mismo mensaje genérico al cliente).
AUTH_MUST_CHANGE_PASSWORD	403	Debe cambiar la contraseña temporal antes de operar.
FORBIDDEN_ROLE	403	El rol no tiene acceso al recurso.
DUPLICATE_SKU / DUPLICATE_DOCUMENT / DUPLICATE_CODE	409	Violación de unicidad (producto, cliente, categoría/unidad).
INVENTORY_INSUFFICIENT_STOCK	422	Stock insuficiente en venta o ajuste negativo.
INVENTORY_REASON_REQUIRED	422	Ajuste manual sin motivo.
QUOTE_CUSTOMER_REQUIRED	422	Cotización con cliente genérico o sin cliente.

errorCode	HTTP	Situación
QUOTE_NOT_CONVERTIBLE	409	Cotización rechazada, vencida o ya convertida.
SALE_GENERIC_CUSTOMER_CREDIT	422	Venta con saldo pendiente usando Público general.
SALE_ALREADY_CANCELLED	409	Venta ya anulada.
SALE_INACTIVE_PRODUCT	422	Producto inactivo en el carrito.
PAYMENT_EXCEEDS_BALANCE	422	Pago mayor al saldo pendiente.
PAYMENT_SALE_CANCELLED	409	Pago sobre venta anulada.
PAYMENT_REFERENCE_REQUIRED	422	Método que exige referencia sin reference_number.
ACCOUNTING_UNBALANCED_ENTRY	500	Asiento descuadrado (error interno: aborta la transacción).

Anexo C. Glosario

Término	Definición en este proyecto
MVP	Producto mínimo viable: alcance definido en las secciones 5 y 6, sin extensiones.
Nota de venta interna (NV)	Documento interno no fiscal que respalda la venta; no es comprobante SUNAT.
Precontabilidad	Asientos automáticos internos para trazabilidad económica; no reemplaza contabilidad tributaria.
Snapshot	Copia de SKU, nombre, unidad y precio del producto guardada en los ítems de cotización/venta.
Baja lógica	Inactivación por status o deleted_at en lugar de borrado físico, para conservar historial.
Movimiento de inventario	Único mecanismo válido para modificar el stock, con previous_stock y new_stock.
Correlativo	Número secuencial único por tipo de documento (COT, NV, PAG, AST) generado en backend.
Cliente genérico	Registro "Público general" usado solo en ventas al contado.
Caso de uso	Clase de negocio que orquesta una operación completa dentro de una transacción.

Anexo D. Referencias del proyecto

- Documento Maestro POS / Gestión Comercial MVP v1.0 (fuente única de verdad funcional).
- schema.prisma y carpeta prisma/migrations (fuente de verdad del modelo de datos una vez iniciado el desarrollo).
- Swagger /docs del backend (fuente de verdad de contratos una vez implementados).
- Los diagramas Mermaid de este documento pueden renderizarse en mermaid.live o en la wiki del repositorio.